Tokenized Deposit/Stablecoin Management System OpenAPI Documentation for Service Provider v2.2.0

This document outlines two methods for Service Providers to access the Tokenized Deposit/Stablecoin Management System: calling the system's OpenAPIs or directly invoking the Token contract.

1. Access By OpenAPIs

1.1 Introduction

1.1.1 Overview

This document introduces the service interfaces for a Service Provider's business system to access the Tokenized Deposit/Stablecoin Management System. Its primary functions encompass wallet activation, Tokenized Deposits/Stablecoins (Tokens) withdrawal, transfers, authorizations, and authorized transfers among other interfaces.

1.1.2 Communication Mode

1. All APIs outlined in this document require HTTPS requests (POST mode) with UTF-8 encoded characters.
2. Outputs are delivered in JSON format.
3. Each request must be signed by the Service Provider. Responses from the Tokenized Deposit/Stablecoin Management System are verifiable by the Service Provider.

1.1.3 Definition of Terms

1. **accessKey**: A credential issued by Tokenized Deposit/Stablecoin Management System Operator to the Service Provider for accessing OpenAPIs.
2. **Bus-Trace-ID**: The unique identifier provided by the Service Provider when calling OpenAPIs and sending requests to the System. It consists of 32 digits or letters, or a combination of both. If the Service Provider calls the same interface multiple times with the same Bus-Trace-ID, only the first request will be processed.
3. **transCode**: The unique identifier provided by the Service Provider when calling OpenAPIs and sending requests to the System for parameter signing. It consists of 32 digits or letters, or a combination of both.
4. **reqTime**: The timestamp (13-digit decimal, millisecond precision) at which the Service Provider sends a request. The System will verify timestamp and reject requests if the time difference between the current System time and the request time exceeds 5 minutes.
5. **Public and private key pairs for signing**: The Service Provider must securely store the private key used for signing requests, while the public key is used by the System for verification.
6. **Public and private key pairs for data encryption/decryption**: When Service Providers making OpenAPI calls, certain sensitive data will be encrypted using the public key. Upon receiving the request, the System will use the corresponding data decryption private key of the Service Provider to decrypt the data.
7. **respTime**: The timestamp of the System's response (13-digit decimal, millisecond precision).

1.1.4 Data Signing and Signature Verification Rules

1. All requests in the OpenAPI are signed and verified by RSA asymmetric encryption.
2. To sign a request, the Service Provider concatenates the values of accessKey, transCode, and reqTime into strings in sequence, converts them to lowercase, and signs them with the private key.
3. In the signature verification of the response information, the original data for the signature is formed by concatenating the values of the request accessKey, the transCode in the response body, the response code, and the respTime in sequence. The concatenated values are then converted to lowercase and signed using the System's private key. The signed data is returned to the Service Provider and requires verification using the provided public key.

1.2 Interface Specification

1.2.1 Request Rules

1. Configure the HTTP request header to include `key = accessKey`, where `value` is the access key provided by the Tokenized Deposit/Stablecoin Management System.
2. Configure the HTTP request header to include `key = Bus-Trace-ID`, where value is the unique identifier for the request, consisting of 32 digits or letters, or a combination of both.
3. Below is an overview of the requests and responses.

1.2.2 Format of the Request Message

Parameter name	Description	Data type	Required
transCode	Unique code of the request	String	true
reqTime	The timestamp (13-digit decimal, millisecond precision) at which the Service Provider makes a request.	Long	true
body	Request body (defined by the specific business)		true
mac	Signature	String	true

Sample Request:

```
{
  "body": {},
  "mac":
"V8mGaMKEcKuqKDMX9VsVOY4HgNZLgFOzhuW8nyLLZFfYvpBwxK8wWMuQqJ2XmSZcUhpyKwvKXfSw8PDOnYT
fsmnz8XEMTZIOpinXhTALobddT4xlMvguMctFpE6W4O4Kzi45kmxwViDKXvJWsPhts7alnZLS+JuCB7rzSYd
/rjA=",
  "reqTime": 1712739465790
}
```

1.2.3 Format of Response Message

Parameter name	Description	Data type	Required
code	Response code	String	true
transCode	Unique code of the request	String	true
respTime	The timestamp (13-digit decimal, millisecond precision) at which the System makes a response.	Long	true
body	Request body (as defined by the specific business)		true
mac	Signature	String	true

Sample Response:


```
{
  "code": "MSG_90_0000",
  "respTime": 1712740004308,
  "transCode": "37e8117701164bcfb8e11f099d201d80",
  "body": [
    {
      "tdCode": "123456789",
      "tdName": "HSC coin",
      "tdSymbol": "HSC",
      "tdStatus": 1,
      "tdChainName": "Hyperledger Besu",
      "tdChainVersion": "",
      "tdChainId": "1337",
      "currencySymbol": "USD",
      "currencyPrice": "1",
      "tdPrice": "1",
      "accessTypeList": [
        1
      ]
    }
  ],
  "mac":
"Thu7/X5yczVG0eHG0ARML8BULJv4vHLcVoyziLn9JxKDOqO3UWRgXAz+RN/yF7592uH7qXGmr1mME7q0NXs
mXkPT/Akt2ETVv1JAFf+nH1ZLexp8LAzW06P9kxDc7XBHuQZ7h4225KH0sgRj4Hrkaoqk4OdYFPr2D2FPAfg
W/20="
}
```


1.2.4 Response Status Code

Response code	Description
MSG_90_0000	Success

Response code	Description
MSG_90_0001	accessToken cannot be empty
MSG_90_0002	Incorrect request format
MSG_90_0003	reqTime cannot be empty
MSG_90_0004	transCode cannot be empty
MSG_90_0005	mac cannot be empty
MSG_90_0006	Invalid access key
MSG_90_0007	Incorrect Body request format
MSG_90_0008	Incorrect reqTime format
MSG_90_0009	Invalid request
MSG_90_0010	Signature verification failed
MSG_90_0011	Incorrect transCode length
MSG_90_0012	Incorrect transCode format. It is composed of English letters or numbers or a combination of them
MSG_90_0013	The system is currently experiencing a high volume of requests. Please try again later.
MSG_90_0014	Incorrect parameter format, failed to convert the type
MSG_90_0015	Bus-Trace-ID cannot be empty
MSG_90_0016	Incorrect Bus-Trace-ID length
MSG_90_0017	Incorrect Bus-Trace-ID format. It is composed of English letters or numbers or a combination of them
MSG_90_1000	Token ID cannot be empty
MSG_90_1001	The Token does not exist or has been deactivated
MSG_90_2000	Wallet address cannot be empty
MSG_90_2001	Wallet type cannot be empty
MSG_90_2002	Wallet address is too long
MSG_90_2003	Wallet alias is too long
MSG_90_2004	Bank account is too long
MSG_90_2005	Bank account type is too long

Response code	Description
MSG_90_2006	Bank payment number cannot be empty
MSG_90_2007	Bank payment number is too long
MSG_90_2008	Bank payment time cannot be empty
MSG_90_2009	Bank payment/withdrawal account cannot be empty
MSG_90_2010	Bank payment/withdrawal account is too long
MSG_90_2011	The purchase/withdrawal amount cannot be empty
MSG_90_2012	The purchase amount must be greater than 1
MSG_90_2013	The purchase/withdrawal amount must be less than 100000000
MSG_90_2014	The query number cannot be empty
MSG_90_2015	The wallet address already exists
MSG_90_2016	Wallet type does not exist
MSG_90_2017	No permission to create the wallet
MSG_90_2018	No permission to create this type of wallet or the wallet type has been deactivated
MSG_90_2019	The wallet address does not exist
MSG_90_2020	The account is not valid yet
MSG_90_2021	There are transactions in recharge or withdrawal pending processing
MSG_90_2022	Invalid wallet address
MSG_90_2023	Insufficient account balance
MSG_90_2024	Recharge or withdrawal information does not exist
MSG_90_2025	The offline calculated hash value does not match the input hash value
MSG_90_2027	Quantity cannot be empty
MSG_90_2028	The original transaction cannot be empty
MSG_90_2029	Signature of the transaction cannot be empty
MSG_90_2030	Transaction hash cannot be empty
MSG_90_2031	Insufficient access permissions
MSG_90_2032	Failed to send the transaction

Response code	Description
MSG_90_2033	Quantity must be greater than 0
MSG_90_2034	EstimateGas failed (failed to create the transaction)
MSG_90_2035	Failed to get transaction count
MSG_90_2036	Transaction does not exist
MSG_90_2037	Please retain two decimal places for the withdrawal quantity
MSG_90_2038	Purchase quantity must be an integer
MSG_90_2039	Access type does not exist
MSG_90_2040	Order does not exist
MSG_90_2040	Order does not exist
MSG_90_2041	The withdrawal amount must be greater than 0
MSG_90_2042	After purchase, the balance will exceed the limitation of this wallet.
MSG_90_2043	Authorization record does not exist
MSG_90_2044	Insufficient authorization amount
MSG_90_2045	The length of integer or decimal places exceeds the limit
MSG_90_2046	The account contains transactions that have been created but not yet sent and have not expired
MSG_90_2047	The transaction does not exist or has expired
MSG_90_2048	The original data cannot be null
MSG_90_2049	Meta transactions are not supported
MSG_90_2050	Invalid method
MSG_90_2051	Public key cannot be empty
MSG_90_2052	Public key conversion failed, please check on the public key format
MSG_90_2053	Public key does not match the wallet address
MSG_90_2054	The cross chain fee does not be same
MSG_90_5000	Wallet is being activated
MSG_90_5001	The wallet has been activated
MSG_90_5002	Purchase information submitted

Response code	Description
MSG_90_5003	MQ is sending an exception
MSG_90_5004	The exchange rate cannot be null
MSG_90_5005	The cross chain fee cannot be null
MSG_90_5006	The blockChain not set cross chain info
MSG_90_5007	Signature verification failed
MSG_90_6027	The approve record cannot be empty
MSG_90_6028	It's not your turn to approve yet
MSG_90_6029	The approve submit type cannot be empty
MSG_90_6030	Approval state error
MSG_90_6031	The current approval is in the first step and cannot be rolled back
MSG_90_7000	Unsigned transaction messages cannot be empty
MSG_90_7001	Kyc required type error
MSG_90_7002	Verifiable credential cannot be empty
MSG_90_7003	No transaction permission
MSG_00_0009	The page information cannot be empty
MSG_00_0010	The number of pages cannot be empty
MSG_00_0011	The size of pages cannot be empty
MSG_00_0012	Decrypt exceptions
MSG_00_0013	Encryption anomalies
MSG_00_0014	Permission verification failed.
MSG_00_0015	Your certificate is invalid or has expired. Please verify if the server has authorized or reapply for the certificate
MSG_00_0100	Network anomaly
MSG_00_0101	Data acquisition exception
MSG_00_0102	Data parsing exception
MSG_00_0103	Business exception
MSG_00_0104	The org info not found

Response code	Description
MSG_00_0105	The TD info not found
MSG_00_0106	The org code cannot be empty
MSG_00_0107	The access token cannot be empty
MSG_00_0108	Access token not exist
MSG_00_0109	Download exception

1.3 Implementation Notes & Known Gotchas

This section records behaviour discovered during integration that differs from or is not covered by the API documentation above. Check here before debugging unexpected errors.

1.3.1 Wallet address normalisation

The API accepts and returns wallet addresses in mixed case, but treats them case-insensitively. **Always normalise addresses to lowercase** before storing, comparing, or sending them. Inconsistent casing is a common source of "wallet does not exist" errors.

1.3.2 Async operations — polling strategy

Several APIs are fire-and-poll:

Operation	Submit endpoint	Poll endpoint	Typical wait
Wallet activation	4.1	4.2	5–30 s
Token purchase	2.1	2.2	5–40 s
Withdrawal	2.3+2.4	2.5	5–30 s
Cross-chain transfer	3.13+3.14	3.15	30–120 s

Recommended poll cadence: every 2 s, give up after 40–60 s for purchases; cross-chain may take up to 2 minutes. Status codes: `20`=Pending, `30`=Submitting, `35`=Successful, `40`=Failed.

1.3.3 One pending transaction per wallet (MSG_90_2046)

Each wallet may have only **one created-but-unsent unexpired transaction at a time**. If you call a Create endpoint and immediately call it again (or call a different Create endpoint for the same wallet), the second call returns `MSG_90_2046`. Wait for the first transaction to either complete, be sent, or expire before creating another.

1.3.4 MSG_90_0013 — system high load

When this code is returned, the request was not processed. Retry with exponential backoff (up to ~5 attempts). Do **not** resend without retrying — the operation was not started.

1.3.5 `Bus-Trace-ID` idempotency

Duplicate `Bus-Trace-ID` values are deduplicated server-side — only the first request with a given ID is processed; subsequent calls with the same ID return the cached result of the first. Always generate a fresh 32-character alphanumeric ID per request. Never reuse IDs across retries of a failed call.

1.4 Transaction Flow Reference

1.4.1 Regular Transaction (Transfer / Approve / Cross-Chain)

```
1. createTransaction  →  rawTransaction, toSignMessage
2. [offline] sign toSignMessage with wallet private key → signedMessage, txHash
   ↳ toSignMessage is either a JSON tx object (legacy RLP + EIP-155) or a hex digest
     Detect by checking if value starts with "{" vs "0x"
3. sendTransaction(rawTransaction, signedMessage, txHash)  →  transactionId
4. queryResult(transactionId)  →  status, txHash
```

1.4.2 Meta Transaction (Gasless)

```
1. createTransaction  →  rawTransaction, toSignMessage, function, gasLimit, doMain,
expireTime, metaNonce
2. [offline] EIP-712 sign using the above params → signedMessage
3. sendTransaction(rawTransaction, signedMessage)  →  transactionId (no txHash
needed)
4. queryResult(transactionId) via regular query APIs
```

1.4.3 Purchase (Top-up) Flow

```
1. buy(bankOrderNumber, bankOrderPayTime, bankOrderPayAccount, tokenCount)  →
orderNumber
   ↳ tokenCount must be a whole integer (no decimals) – MSG_90_2038 otherwise
2. buy/result(orderNumber)  →  status (poll every ~2 s until status = 35 or 40)
   ↳ On status = 40 (Failed): reverse any local fiat deduction
```

1.4.4 Withdrawal Flow

```
Regular:
1. withdrawal/createTransaction  →  rawTransaction, toSignMessage
2. [offline] sign → signedMessage, txHash
3. withdrawal/sendTransaction(rawTransaction, signedMessage, txHash,
bankOrderNumber, bankOrderWithdrawAccount)  →  orderNumber
4. withdrawal/result(orderNumber)  →  status

Meta:
1. metaTransaction/withdrawal/createTransaction  →  meta params
2. [offline] EIP-712 sign → signedMessage
3. metaTransaction/withdrawal/sendTransaction(rawTransaction, signedMessage,
bankOrderNumber, bankOrderWithdrawAccount)  →  orderNumber
4. withdrawal/result(orderNumber)  →  status
```

1.4.5 Wallet Activation Flow

```
1. wallet/create(walletAddress, walletType, ...)  →  status=0 (Activating)
2. wallet/create/result(walletAddress)  →  status (poll until status = 1)
```

1.5 Interface List

1.5.1 Service Provider Module

1.5.1.1 Get Access Permission Information

url: `/api/openapi/v1/sp/access/auth/get`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

The interface is called to obtain Token permission information by Token ID or access type.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "accessType": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSpAccessTokenAuthReqVO	ApiReqInfoSpAccessTokenAuthReqVO	body	true	ApiReqInfoSpAccessTokenAuthReqVO	ApiReqInfoSpAccessTokenAuthReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SpAccessTokenAuthReqVO	SpAccessTokenAuthReqVO
tokenCode	Token Code		true	string	
accessType	Access Type 1: API 5: Contract 10: Chain		false	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoListSpAccessTokenAuthRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		array	SpAccessTokenAuthRespVO
tokenCode	Token Code	string	
tokenName	Token Name	string	
tokenStatus	Token Status 1：Enable 2：Disable	integer(int32)	
accessType	Access Type 1: API 5: Contract 10: Chain	integer(int32)	
contractAddress	Contract Address	string	

name	description	type	schema
manageWalletAddress	Manage Wallet Address	string	
authList	Auth List	array	SpAccessTdAuthFuncRespVO
accessName	Name	string	
apiFuncUrl	If the parameter "Api Func Url" is sent by the OpenAPI, its value is the request url of the OpenAPI.	string	
contractFuncName	If the parameter "Contract Func Name" is sent by a contract or a chain node, its value is the function name in the contract which accesses the Token contract.	string	
funcParam	Func Param	array	SpAccessTokenAuthFuncParamRespVO
walletType	Wallet Type	integer(int64)	
walletTypeName	Wallet Type Name	string	
maxWalletBalance	Max Wallet Balance	number	
maxTxCountDaily	Max Tx Count Daily	number	
maxTxCountPer	Max Tx Count Per	number	
status	Wallet Type Status 10:Enable 15:Disable	integer(int32)	
tokenType	Token Type: 1=stablecoin 5=tokenized deposit 20：Tokenized MMF	integer(int32)	
mac		string	

Response Example:

```
{
    "code": "",
```

```
    "respTime": 0,
    "transCode": "",
    "body": [
        {
            "tokenCode": "",
            "tokenName": "",
            "tokenStatus": 0,
            "accessType": 0,
            "contractAddress": "",
            "manageWalletAddress": "",
            "authList": [
                {
                    "accessName": "",
                    "apiFuncUrl": "",
                    "contractFuncName": "",
                    "funcParam": [
                        {
                            "walletType": 0,
                            "walletTypeName": "",
                            "maxWalletBalance": 0,
                            "maxTxCountDaily": 0,
                            "maxTxCountPer": 0,
                            "status": 0
                        }
                    ]
                }
            ],
            "tokenType": 0
        }
    ],
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```

1.5.1.2 Query Accessed Tokens Information

url: `/api/openapi/v1/sp/access/token/get`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called to retrieve accessed Tokens information, including its basic details and access methods.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {},
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfo	ApiReqInfo	body	true	ApiReqInfo	ApiReqInfo
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	object	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoListSpAccessTokenRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		array	SpAccessTokenRespVO
tokenCode	Token Code	string	
tokenName	Token Name	string	
tokenSymbol	Token Symbol	string	
tokenDecimalPrecision	Token Decimal Precision	integer(int32)	
tokenStatus	Token Status 1: Enable 2: Disable	integer(int32)	
tokenChainName	Token Chain Name	string	
tokenChainVersion	Token Chain Version	string	
tokenChainId	Token Chain Id	string	
currencySymbol	Pegged Currency	string	
currencyPrice	Currency Amount	number	
tokenPrice	Token Price	number	
accessTypeList	Access Type 1: API 5: Contract 10: Chain	array	integer(int32)
tokenType	Token Type 1: stablecoin 5: tokenized deposit 20: Tokenized MMF	integer(int32)	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": [
```

```
        {
            "tokenCode": "",
            "tokenName": "",
            "tokenSymbol": "",
            "tokenDecimalPrecision": 0,
            "tokenStatus": 0,
            "tokenChainName": "",
            "tokenChainVersion": "",
            "tokenChainId": "",
            "currencySymbol": "",
            "currencyPrice": 0,
            "tokenPrice": 0,
            "accessTypeList": [],
            "tokenType": 0
        }
    ],
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.1.3 Update Access Key

url: `/api/openapi/v1/sp/access/key/update`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called to update the access key for accessing the Tokenized Deposit/Stablecoin Management System. After the update, all new requests should be sent by the latest access key.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {},
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfo	ApiReqInfo	body	true	ApiReqInfo	ApiReqInfo
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	object	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoRefreshAccessKeyRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		RefreshAccessKeyRespVO	RefreshAccessKeyRespVO
accessKey	Access Key	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "accessKey": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.1.4 Get available cross-chain information for the connected token

url: `/api/openapi/v1/sp/access/crossChain/get`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

By calling this interface, you can get the latest exchange rate of the specified currency.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {},
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfo	ApiReqInfo	body	true	ApiReqInfo	ApiReqInfo
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	object	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoListTokenCrossChainRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		array	TokenCrossChainRespVO
sendTokenCode	Send Token Code	string	
sendTokenName	Send Token Name	string	
sendTokenSymbol	Send Token Symbol	string	
sendCurrencySymbol	Send Token Pegged Currency	string	
sendBlockchainName	Send Token Block Chain Name	string	
receiveTokenCode	Send Token Code	string	
receiveTokenName	Receive Token Name	string	
receiveTokenSymbol	Receive Token Symbol	string	
receiveCurrencySymbol	Receive Token Pegged Currency	string	

name	description	type	schema
receiveBlockchainName	Receive Token Block Chain Name	string	
crossChainFee	Cross Chain Service Fee(单笔交易从发起账户扣除的跨链服务费)	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": [
        {
            "sendTokenCode": "",
            "sendTokenName": "",
            "sendTokenSymbol": "",
            "sendCurrencySymbol": "",
            "sendBlockchainName": "",
            "receiveTokenCode": "",
            "receiveTokenName": "",
            "receiveTokenSymbol": "",
            "receiveCurrencySymbol": "",
            "receiveBlockchainName": "",
            "crossChainFee": ""
        }
    ],
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2 Token Order Module

1.5.2.1 Purchase Token

url: `/api/openapi/v1/token/manage/buy`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used for users to initiate a Token transaction and upload the parameters, including the wallet address, bank order number, payment account, transaction amount and time.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "bankOrderNumber": "",
    "bankOrderPayTime": 0,
    "bankOrderPayAccount": "",
    "tokenCount": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenBuyReqVO	ApiReqInfoTokenBuyReqVO	body	true	ApiReqInfoTokenBuyReqVO	ApiReqInfoTokenBuyReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	TokenBuyReqVO	TokenBuyReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
bankOrderNumber	Bank Order Number (Length must be between 1 and 100)		true	string	
bankOrderPayTime	Bank Order Pay Time		true	integer(int64)	
bankOrderPayAccount	Bank Order Pay Account (Length must be between 1 and 100)		true	string	
tokenCount	Amount		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoTokenBuyRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		TokenBuyRespVO	TokenBuyRespVO
tokenCode	Token Code	string	
walletAddress	Wallet Address	string	
bankOrderNumber	Bank Order Number	string	
bankOrderPayTime	Bank Order Pay Time	integer(int64)	
bankOrderPayAccount	Bank Order Pay Account	string	
tokenCount	Amount	number	
orderNumber	Order Number	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
transCode	Request Trans Code	string	
mac		string	

Response Example:

```
{
```

```
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "bankOrderNumber": "",
        "bankOrderPayTime": 0,
        "bankOrderPayAccount": "",
        "tokenCount": 0,
        "orderNumber": "",
        "status": 0,
        "txHash": "",
        "txTime": 0,
        "transCode": ""
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.2.2 Purchase Token - Query Result

url: `/api/openapi/v1/token/manage/buy/result`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when user retrieves the result of Token purchase application.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "orderNumber": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenBuyResultReqVO	ApiReqInfoTokenBuyResultReqVO	body	true	ApiReqInfoTokenBuyResultReqVO	ApiReqInfoTokenBuyResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	TokenBuyResultReqVO	TokenBuyResultReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
orderNumber	Order Number		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoTokenBuyRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		TokenBuyRespVO	TokenBuyRespVO
tokenCode	Token Code	string	
walletAddress	Wallet Address	string	
bankOrderNumber	Bank Order Number	string	
bankOrderPayTime	Bank Order Pay Time	integer(int64)	

name	description	type	schema
bankOrderPayAccount	Bank Order Pay Account	string	
tokenCount	Amount	number	
orderNumber	Order Number	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
transCode	Request Trans Code	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "bankOrderNumber": "",
        "bankOrderPayTime": 0,
        "bankOrderPayAccount": "",
        "tokenCount": 0,
        "orderNumber": "",
        "status": 0,
        "txHash": "",
        "txTime": 0,
        "transCode": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2.3 Withdrawal - Create Transaction

url: `/api/openapi/v1/token/manage/withdrawal/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user withdraws Tokens.

Step 1: The user calls the "Withdraw Token - Create Transaction" interface to compile the data into the transaction body, including the Token ID, wallet address, and withdrawal amount.

Step 2: The user uses the wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Withdraw Token - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Withdrawal Token - Query Result" interface to query the withdrawal results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "tokenCount": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateTokenWithdrawReqVO	ApiReqInfoCreateTokenWithdrawReqVO	body	true	ApiReqInfoCreateTokenWithdrawReqVO	ApiReqInfoCreateTokenWithdrawReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateTokenWithdrawReqVO	CreateTokenWithdrawReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
tokenCount	Withdrawal Amount		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2.4 Withdrawal - Send Transaction

url: `/api/openapi/v1/token/manage/withdrawal/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user withdraws Tokens.

Step 1: The user calls the "Withdraw Token - Create Transaction" interface to compile the data into the transaction body, including the Token ID, wallet address, and withdrawal amount.

Step 2: The user uses the wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Withdraw Token - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Withdrawal Token - Query Result" interface to query the withdrawal results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "bankOrderNumber": "",
    "bankOrderWithdrawAccount": "",
    "tokenCount": 0,
    "rawTransaction": "",
    "signedMessage": "",
```

```
    "txHash": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendTokenWithdrawReqVO	ApiReqInfoSendTokenWithdrawReqVO	body	true	ApiReqInfoSendTokenWithdrawReqVO	ApiReqInfoSendTokenWithdrawReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendTokenWithdrawReqVO	SendTokenWithdrawReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
bankOrderNumber	Bank Order Number (Length must be between 1 and 100)		true	string	
bankOrderWithdrawAccount	Bank Order Withdraw Account (Length must be between 1 and 100)		true	string	
tokenCount	Withdrawal Amount		true	number	
rawTransaction	Original Transaction Message		true	string	
signedMessage	Sign Message		true	string	
txHash	Tx Hash		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoTokenWithdrawalRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		TokenWithdrawalRespVO	TokenWithdrawalRespVO
tokenCode	Token Code	string	
walletAddress	Wallet Address	string	
tokenCount	Amount	number	
orderNumber	Order Number	string	
bankOrderNumber	Bank Order Number	string	
bankOrderWithdrawAccount	Bank Order Pay Account	string	

name	description	type	schema
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
transCode	Request Trans Code	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "tokenCount": 0,
        "orderNumber": "",
        "bankOrderNumber": "",
        "bankOrderWithdrawAccount": "",
        "status": 0,
        "txHash": "",
        "txTime": 0,
        "transCode": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2.5 Withdrawal - Query Result

url: `/api/openapi/v1/token/manage/withdrawal/result`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user withdraws Tokens.

Step 1: The user calls the "Withdraw Token - Create Transaction" interface to compile the data into the transaction body, including the Token ID, wallet address, and withdrawal amount.

Step 2: The user uses the wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Withdraw Token - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Withdrawal Token - Query Result" interface to query the withdrawal results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "orderNumber": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenWithdrawalResultReqVO	ApiReqInfoTokenWithdrawalResultReqVO	body	true	ApiReqInfoTokenWithdrawalResultReqVO	ApiReqInfoTokenWithdrawalResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	TokenWithdrawalResultReqVO	TokenWithdrawalResultReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
orderNumber	Order Number		true	string	
mac	Signature		true	string	

Status:

code	description	schema

code	description	schema
200	OK	ApiRespInfoTokenWithdrawalRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		TokenWithdrawalRespVO	TokenWithdrawalRespVO
tokenCode	Token Code	string	
walletAddress	Wallet Address	string	
tokenCount	Amount	number	
orderNumber	Order Number	string	
bankOrderNumber	Bank Order Number	string	
bankOrderWithdrawAccount	Bank Order Pay Account	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
transCode	Request Trans Code	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "tokenCount": 0,
        "orderNumber": "",
        "bankOrderNumber": "",
        "bankOrderWithdrawAccount": "",
        "status": 0,
```

```
        "txHash": "",
        "txTime": 0,
        "transCode": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2.6 Query Top-up-Withdrawal Details by Pagination

url: `/api/openapi/v1/token/manage/query`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used for users to query Token withdrawals information by various optional parameters.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "walletAddress": "",
      "bankOrderNumber": "",
      "orderNumber": "",
      "status": 0,
      "type": 0,
```

```
      "txHash": "",
      "startTxTime": 0,
      "endTxTime": 0,
      "startCreateTime": 0,
      "endCreateTime": 0
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```


Params:

name	description	in	require	type	schema
apiReqInfoDataTableTokenManageReqVO	ApiReqInfoDataTableTokenManageReqVO	body	true	ApiReqInfoDataTableTokenManageReqVO	ApiReqInfoDataTableTokenManageReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableTokenManageReqVO	DataTableTokenManageReqVO
data			true	TokenManageReqVO	TokenManageReqVO
tokenCode	Token Code		false	string	
walletAddress	Wallet Address		false	string	
bankOrderNumber	Bank Order Number		false	string	
orderNumber	Order Number		false	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed		false	integer(int32)	
type	Type 1:Top-up 2:Withdrawal		false	integer(int32)	
txHash	Tx Hash		false	string	
startTxTime	Start Tx Time		false	integer(int64)	
endTxTime	End Tx Time		false	integer(int64)	
startCreateTime	Start Create Time		false	integer(int64)	
endCreateTime	End Create Time		false	integer(int64)	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataTokenManageRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataTokenManageRespVO	ResultDataTokenManageRespVO

name	description	type	schema
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	TokenManageRespVO
tokenCode	Tokenized Deposit Code	string	
tokenName	Tokenized Deposit Name	string	
walletAddress	Wallet Address	string	
bankOrderNumber	Bank Order Number	string	
tokenCount	Top-up Amount	number	
orderNumber	Order Number	string	
status	status 20：Pending submission 30：Submitting 35：Purchase/Withdrawal successful 40：Purchase/Withdrawal failed	integer(int32)	
type	Type 1:Top-up 2:Withdrawal	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
createTime	Create Time	integer(int64)	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "tokenCode": "",
                "tokenName": "",
                "walletAddress": "",
                "bankOrderNumber": "",
                "tokenCount": 0,
                "orderNumber": "",
                "status": 0,
                "type": 0,
                "txHash": "",
                "txTime": 0,
                "createTime": 0
            }
        ]
    },
    "mac": ""
```

```
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.2.7 Query Top-up-Withdrawal Details

url: `/api/openapi/v1/token/manage/detail`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called to query Token withdrawals information by Token ID and order number.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "orderNumber": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenManageDetailReqVO	ApiReqInfoTokenManageDetailReqVO	body	true	ApiReqInfoTokenManageDetailReqVO	ApiReqInfoTokenManageDetailReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	TokenManageDetailReqVO	TokenManageDetailReqVO
tokenCode	Token Code		true	string	
orderNumber	Order Number		true	string	

name	description	in	require	type	schema
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoTokenManageDetailRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		TokenManageDetailRespVO	TokenManageDetailRespVO
tokenCode	Token Code	string	
tokenName	Tokenized Deposit Name	string	
walletAddress	Wallet Address	string	
walletName	Wallet Alias	string	
bankOrderNumber	Bank Order Number	string	
bankOrderPayTime	Bank Order Pay Time	string	
bankOrderPayAccount	Bank Order Pay Account	string	
tokenCount	Amount	number	
orderNumber	Order Number	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
type	Type 1: Top-up 2: Withdrawal	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
createTime	Create Time	integer(int64)	
transCode	Request Trans Code	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "tokenName": "",
        "walletAddress": "",
        "walletName": "",
        "bankOrderNumber": "",
        "bankOrderPayTime": "",
        "bankOrderPayAccount": "",
        "tokenCount": 0,
        "orderNumber": "",
        "status": 0,
        "type": 0,
        "txHash": "",
        "txTime": 0,
        "createTime": 0,
        "transCode": ""
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.3 Token Transaction Module

1.5.3.1 Transfer - Create Transaction

url: `/api/openapi/v1/token/transaction/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Transfer - Query Transaction Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateTransferTransactionReqVO	ApiReqInfoCreateTransferTransactionReqVO	body	true	ApiReqInfoCreateTransferTransactionReqVO	ApiReqInfoCreateTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateTransferTransactionReqVO	CreateTransferTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.2 Transfer - Send Transaction

url: `/api/openapi/v1/token/transaction/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Transfer - Query Transaction Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "rawTransaction": "",
    "signedMessage": "",
    "txHash": "",
    "remark": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendTransferTransactionReqVO	ApiReqInfoSendTransferTransactionReqVO	body	true	ApiReqInfoSendTransferTransactionReqVO	ApiReqInfoSendTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendTransferTransactionReqVO	SendTransferTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
rawTransaction	Original Transaction Message		true	string	
signedMessage	Sign Message		true	string	

name	description	in	require	type	schema
txHash	Tx Hash		true	string	
remark	Remark		false	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendTransferTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendTransferTransactionRespVO	SendTransferTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.3 Transfer - Query Transaction Result

url: `/api/openapi/v1/token/transaction/transfer/queryResult`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Transfer - Query Transaction Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "transactionId": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoQueryTransactionResultReqVO	ApiReqInfoQueryTransactionResultReqVO	body	true	ApiReqInfoQueryTransactionResultReqVO	ApiReqInfoQueryTransactionResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	QueryTransactionResultReqVO	QueryTransactionResultReqVO
transactionId	Transaction Id		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoQueryTransactionResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		QueryTransactionResultRespVO	QueryTransactionResultRespVO
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
fromWalletAddress	From Wallet Address	string	
toWalletAddress	To Wallet Address	string	
count	Count	number	
transCode	Trans Code	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "txHash": "",
        "txTime": 0,
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "count": 0,
        "transCode": "",
        "status": 0
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.4 Query transfer record of the wallet

url: `/api/openapi/v1/token/transaction/query/transferList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used to query wallet transfer records, which can get all transfer records with sender wallet addresses, from wallet addresses, and to wallet addresses as input.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "walletAddress": "",
      "tokenCode": ""
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableQueryTransferListReqVO	ApiReqInfoDataTableQueryTransferListReqVO	body	true	ApiReqInfoDataTableQueryTransferListReqVO	ApiReqInfoDataTableQueryTransferListReqVO
reqTime	Request Time		true	integer(int64)	

name	description	in	require	type	schema
transCode	Request ID		true	string	
body			false	DataTableQueryTransferListReqVO	DataTableQueryTransferListReqVO
data			true	QueryTransferListReqVO	QueryTransferListReqVO
walletAddress	Wallet Address		true	string	
tokenCode	Token Code		true	string	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataQueryTransferListRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataQueryTransferListRespVO	ResultDataQueryTransferListRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	QueryTransferListRespVO
senderWalletAddress	Submitter's wallet address	string	
fromWalletAddress	Sender's Wallet Address	string	
toWalletAddress	Receiver's Wallet Address	string	
fromWalletBalance	Sender's Wallet Balance	number	
toWalletBalance	Receiver's Wallet Balance	number	
count	Count	number	
type	type 1: Normal transfer 2: Authorized transfer	integer(int32)	
dataSourcesType	Data Sources Type 1: OpenAPI 2: contract 3: node	integer(int32)	
transCode	Trans Code	string	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	

name	description	type	schema
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "senderWalletAddress": "",
                "fromWalletAddress": "",
                "toWalletAddress": "",
                "fromWalletBalance": 0,
                "toWalletBalance": 0,
                "count": 0,
                "type": 0,
                "dataSourcesType": 0,
                "transCode": "",
                "txHash": "",
                "txTime": 0
            }
        ]
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```

1.5.3.5 Authorization of Increasing Amount - Create Transaction

url: `/api/openapi/v1/token/transaction/approve/increase/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user increases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Increasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Increasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveIncreaseTransactionReqVO	ApiReqInfoCreateApproveIncreaseTransactionReqVO	body	true	ApiReqInfoCreateApproveIncreaseTransactionReqVO	ApiReqInfoCreateApproveIncreaseTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveIncreaseTransactionReqVO	CreateApproveIncreaseTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address		true	string	
toWalletAddress	Delegated Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.6 Authorization of Increasing Amount - Send Transaction

url: `/api/openapi/v1/token/transaction/approve/increase/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user increases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Increasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Increasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "rawTransaction": "",
    "signedMessage": "",
    "txHash": "",
    "remark": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendApproveIncreaseTransactionReqVO	ApiReqInfoSendApproveIncreaseTransactionReqVO	body	true	ApiReqInfoSendApproveIncreaseTransactionReqVO	ApiReqInfoSendApproveIncreaseTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveIncreaseTransactionReqVO	SendApproveIncreaseTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address		true	string	
toWalletAddress	Delegated Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
rawTransaction	Original Transaction Message		true	string	
signedMessage	Sign Message		true	string	
txHash	Tx Hash		true	string	
remark	Remark		false	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveIncreaseTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveIncreaseTransactionRespVO	SendApproveIncreaseTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.7 Authorization of Decreasing Amount - Create Transaction

url: `/api/openapi/v1/token/transaction/approve/decrease/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user decreases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Decreasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Decreasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveDecreaseTransactionReqVO	ApiReqInfoCreateApproveDecreaseTransactionReqVO	body	true	ApiReqInfoCreateApproveDecreaseTransactionReqVO	ApiReqInfoCreateApproveDecreaseTransactionReqVO

name	description	in	require	type	schema
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveDecreaseTransactionReqVO	CreateApproveDecreaseTransactionReqVO
tokenCode	Token Code		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Delegated Wallet Address: the receiver of transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
count	Count		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.8 Authorization of Decreasing Amount - Send Transaction

url: `/api/openapi/v1/token/transaction/approve/decrease/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user decreases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Decreasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Decreasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "rawTransaction": "",
    "signedMessage": "",
    "txHash": "",
```

```
    "remark": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendApproveDecreaseTransactionReqVO	ApiReqInfoSendApproveDecreaseTransactionReqVO	body	true	ApiReqInfoSendApproveDecreaseTransactionReqVO	ApiReqInfoSendApproveDecreaseTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveDecreaseTransactionReqVO	SendApproveDecreaseTransactionReqVO
tokenCode	Token Code		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Delegated Wallet Address: the receiver of transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
count	Count		true	number	
rawTransaction	Original Transaction Message		true	string	
signedMessage	Sign Message		true	string	
txHash	Tx Hash		true	string	
remark	Remark		false	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveDecreaseTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveDecreaseTransactionRespVO	SendApproveDecreaseTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.9 Authorization - Query Authorization Result

url: `/api/openapi/v1/token/transaction/approve/queryResult`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user queries the authorization result. First, the user calls the "Authorization of Increasing Amount - Create Transaction" interface to assemble the data into the transaction body, and signs the transaction by the private key of the from wallet and calculate the offline transaction hash. Then, the user calls the "Authorization of Increasing Amount - Send Transaction" interface to upload the transaction to the blockchain. Finally, the user calls "Authorization - Query Transaction Result" interface to query the authorization result by the transaction hash.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "transactionId": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoQueryApproveResultReqVO	ApiReqInfoQueryApproveResultReqVO	body	true	ApiReqInfoQueryApproveResultReqVO	ApiReqInfoQueryApproveResultReqVO

name	description	in	require	type	schema
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	QueryApproveResultReqVO	QueryApproveResultReqVO
transactionId	Transaction Id		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoQueryApproveResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		QueryApproveResultRespVO	QueryApproveResultRespVO
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
fromWalletAddress	Authorizer's Wallet Address	string	
toWalletAddress	Delegated Wallet Address	string	
count	Count	number	
transCode	Trans Code	string	
status	Status 20：Pending submission 30：Submitting 35：Submitted 40：Submission failed	integer(int32)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
```

```
        "txHash": "",
        "txTime": 0,
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "count": 0,
        "transCode": "",
        "status": 0
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.3.10 Authorized Transfer - Create Transaction

url: `/api/openapi/v1/token/transaction/approve/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers the authorized Tokens.

Step 1: The user calls the "Authorized Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, spender wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorized Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorized Transfer - Query Transaction Result" interface to query the transfer result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "spenderWalletAddress": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveTransferTransactionReqVO	ApiReqInfoCreateApproveTransferTransactionReqVO	body	true	ApiReqInfoCreateApproveTransferTransactionReqVO	ApiReqInfoCreateApproveTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveTransferTransactionReqVO	CreateApproveTransferTransactionReqVO
spenderWalletAddress	Delegated Wallet Address: the sender with transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Receiver's Wallet Address		true	string	
count	Amount		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	

name	description	type	schema
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.11 Authorized Transfer - Send Transaction

url: `/api/openapi/v1/token/transaction/approve/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers the authorized Tokens.

Step 1: The user calls the "Authorized Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, sender wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorized Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorized Transfer - Query Transaction Result" interface to query the transfer result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "spenderWalletAddress": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "signedMessage": "",
    "txHash": "",
    "remark": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendApproveTransferTransactionReqVO	ApiReqInfoSendApproveTransferTransactionReqVO	body	true	ApiReqInfoSendApproveTransferTransactionReqVO	ApiReqInfoSendApproveTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveTransferTransactionReqVO	SendApproveTransferTransactionReqVO
spenderWalletAddress	Delegated Wallet Address: the sender with transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Receiver's Wallet Address		true	string	
count	Amount		true	number	
tokenCode	Token Code		true	string	
signedMessage	Sign Message		true	string	
txHash	Tx Hash		true	string	
remark	Remark		false	string	
rawTransaction	Raw Message		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveTransferTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveTransferTransactionRespVO	SendApproveTransferTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.12 Authorized Transfer - Query Transaction Result

url: `/api/openapi/v1/token/transaction/approve/transfer/queryResult`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user queries the authorized Token transfer result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "transactionId": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoQueryApproveTransactionResultReqVO	ApiReqInfoQueryApproveTransactionResultReqVO	body	true	ApiReqInfoQueryApproveTransactionResultReqVO	ApiReqInfoQueryApproveTransactionResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	QueryApproveTransactionResultReqVO	QueryApproveTransactionResultReqVO
transactionId	Transaction Id		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoQueryApproveTransactionResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		QueryApproveTransactionResultRespVO	QueryApproveTransactionResultRespVO
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
spenderWalletAddress	Delegated Wallet Address	string	
fromWalletAddress	Authorizer's Wallet Address	string	
toWalletAddress	Receiver's Wallet Address	string	
count	Count	number	
transCode	Trans Code	string	

name	description	type	schema
status	Status 20：Pending submission 30：Submitting 35：Submitted 40：Submission failed	integer(int32)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "txHash": "",
        "txTime": 0,
        "spenderWalletAddress": "",
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "count": 0,
        "transCode": "",
        "status": 0
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.13 Cross Chain Transfer - Create Transaction

url: `/api/openapi/v1/token/transaction/crossChain/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user cross chain transfers TD.

Step 1: The user calls the "Cross Chain Transfer - Create Transaction" interface to assemble the data, including the Token ID, from wallet address, to wallet address, and transfer amount into the transaction body.

Step 2: The user uses the private key of the from wallet to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Cross Chain Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: the user calls "Cross Chain Transfer - Query Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "fromTokenCount": 0,
    "toTokenCount": 0,
    "fromTokenCode": "",
    "toTokenCode": "",
    "exchangeRate": 0,
    "crossChainFee": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateCrossChainTransferTransactionReqVO	ApiReqInfoCreateCrossChainTransferTransactionReqVO	body	true	ApiReqInfoCreateCrossChainTransferTransactionReqVO	ApiReqInfoCreateCrossChainTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateCrossChainTransferTransactionReqVO	CreateCrossChainTransferTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
fromTokenCount	From Token Count		true	number	
toTokenCount	To Token Count		true	number	
fromTokenCode	From Token Code		true	string	
toTokenCode	To Token Code		true	string	
exchangeRate	Exchange Rate		true	number	
crossChainFee	Cross Chain Fee		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateTransactionRespVO

code	description	schema
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateTransactionRespVO	CommonCreateTransactionRespVO
rawTransaction	Original Transaction Message	string	
toSignMessage	Message to Sign	string	
chainId	Chain ID	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "rawTransaction": "",
        "toSignMessage": "",
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.14 Cross Chain Transfer - Send Transaction

url: `/api/openapi/v1/token/transaction/crossChain/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user cross chain transfers TD.

Step 1: The user calls the "Cross Chain Transfer - Create Transaction" interface to assemble the data, including the Token ID, from wallet address, to wallet address, and transfer amount into the transaction body.

Step 2: The user uses the private key of the from wallet to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Cross Chain Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: the user calls "Cross Chain Transfer - Query Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "fromTokenCount": 0,
    "toTokenCount": 0,
    "fromTokenCode": "",
    "toTokenCode": "",
    "exchangeRate": 0,
    "crossChainFee": 0,
    "rawTransaction": "",
    "signedMessage": "",
    "txHash": "",
    "remark": ""
  },
  "mac": ""
```

```
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendCrossChainTransferTransactionReqVO	ApiReqInfoSendCrossChainTransferTransactionReqVO	body	true	ApiReqInfoSendCrossChainTransferTransactionReqVO	ApiReqInfoSendCrossChainTransferTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendCrossChainTransferTransactionReqVO	SendCrossChainTransferTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
fromTokenCount	From Token Count		true	number	
toTokenCount	To Token Count		true	number	
fromTokenCode	From Token Code		true	string	
toTokenCode	To Token Code		true	string	
exchangeRate	Exchange Rate		true	number	
crossChainFee	Cross Chain Fee		true	number	
rawTransaction	Original Transaction Message		true	string	
signedMessage	Sign Message		true	string	
txHash	Tx Hash		true	string	
remark	Remark		false	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendCrossChainTransferTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendCrossChainTransferTransactionRespVO	SendCrossChainTransferTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.3.15 Cross Chain Transfer - Query Result

url: `/api/openapi/v1/token/transaction/crossChain/transfer/queryResult`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user cross chain transfers TD.

Step 1: The user calls the "Cross Chain Transfer - Create Transaction" interface to assemble the data, including the Token ID, from wallet address, to wallet address, and transfer amount into the transaction body.

Step 2: The user uses the private key of the from wallet to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Cross Chain Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: the user calls "Cross Chain Transfer - Query Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "transactionId": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoQueryCrossChainTransactionResultReqVO	ApiReqInfoQueryCrossChainTransactionResultReqVO	body	true	ApiReqInfoQueryCrossChainTransactionResultReqVO	ApiReqInfoQueryCrossChainTransactionResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	QueryCrossChainTransactionResultReqVO	QueryCrossChainTransactionResultReqVO
transactionId	Transaction Id		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoQueryCrossChainTransactionResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		QueryCrossChainTransactionResultRespVO	QueryCrossChainTransactionResultRespVO
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
fromWalletAddress	From Wallet Address	string	
fromTokenCode	From Token Code	string	
fromTokenName	From Token Name	string	
fromTokenSymbol	From Token Symbol	string	
toWalletAddress	To Wallet Address	string	
toTokenCode	To Token Code	string	
toTokenName	To Token Name	string	
toTokenSymbol	To Token Symbol	string	
fromTokenCount	From Token Count	number	
crossChainFee	Cross Chain Fee	number	
toTokenCount	To Token Count	number	
transactionId	Transaction Id	string	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
exchangeRate	Exchange Rate	number	
transferCode	Transfer Code	string	
detailList	Cross Chain Transaction Detail List	array	CrossChainTransactionsDetailsRespVO
tokenCode	Tokenized Deposit Code	string	
tokenName	Token Name	string	
tokenSymbol	Token Symbol	string	

name	description	type	schema
tokenCurrencySymbol	Token Currency Symbol	string	
blockCode	Block Code	string	
blockName	Block Name	string	
blockShortName	Block Short Name	string	
detailType	Detail Type 1:Source chain initiates a tranfer. 5：Source chain verifies the transaction. 10：Verify exchange rate 15：Submit verification results 20：Target chain initiates a tranfer 25：Source chain initiates a refund	integer(int32)	
txHash	Tx Hash	string	
txTime	Tx Time	integer(int64)	
txStatus	Tx Status 20：Pending submission 30：Submitting 35：Submission successful 40：Submission failed	integer(int32)	
senderWalletAddress	Sender Wallet Address	string	
fromWalletAddress	From Wallet Address	string	
toWalletAddress	To Wallet Address	string	
tokenCount	Token Count	number	
createTime	Create Time	integer(int64)	
txChainType	Tx Chain Type 0：Exchange rate service 1：Source chain 2：Target chain	integer(int32)	
handlerCode	Handler Code	string	
remarks	Remarks	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "txHash": "",
        "txTime": 0,
        "fromWalletAddress": "",
        "fromTokenCode": "",
        "fromTokenName": "",
        "fromTokenSymbol": "",
        "toWalletAddress": "",
```

```
        "toTokenCode": "",
        "toTokenName": "",
        "toTokenSymbol": "",
        "fromTokenCount": 0,
        "crossChainFee": 0,
        "toTokenCount": 0,
        "transactionId": "",
        "status": 0,
        "exchangeRate": 0,
        "transferCode": "",
        "detailList": [
            {
                "tokenCode": "",
                "tokenName": "",
                "tokenSymbol": "",
                "tokenCurrencySymbol": "",
                "blockCode": "",
                "blockName": "",
                "blockShortName": "",
                "detailType": 0,
                "txHash": "",
                "txTime": 0,
                "txStatus": 0,
                "senderWalletAddress": "",
                "fromWalletAddress": "",
                "toWalletAddress": "",
                "tokenCount": 0,
                "createTime": 0,
                "txChainType": 0,
                "handlerCode": "",
                "remarks": ""
            }
        ]
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```

1.5.4 Token Wallet Module

1.5.4.1 Activate Wallet

url: `/api/openapi/v1/token/wallet/create`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used to activate a Token wallet by uploading the wallet address, wallet name, wallet type, bank card number, card type and Token ID associated with the wallet.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "publicKey": "",
    "walletType": 0,
    "walletName": "",
    "bankCard": "",
    "bankCardType": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoWalletCreateReqVO	ApiReqInfoWalletCreateReqVO	body	true	ApiReqInfoWalletCreateReqVO	ApiReqInfoWalletCreateReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	WalletCreateReqVO	WalletCreateReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
publicKey	Public Key(Hex)		false	string	
walletType	Wallet Type		true	integer(int64)	
walletName	Wallet Alias【Length must be between 1 and 100.】		false	string	
bankCard	Bank Card【Length must be between 1 and 100.】		false	string	
bankCardType	Bank Card Type【Length must be between 1 and 100.】		false	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoWalletCreateResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		WalletCreateResultRespVO	WalletCreateResultRespVO
tokenCode	Tokenized Deposit Code	string	
walletAddress	Wallet Address	string	
walletType	Wallet Type	integer(int64)	
walletName	Wallet Alias	string	
bankCard	Bank Card	string	
bankCardType	Bank Card Type	string	
status	Status 0:Activating 1：Enable 2：Disable 3：Deactivate	integer(int32)	
createTime	Create Time	integer(int64)	
createTxHash	Create Tx Hash	string	
createTxTime	Create Tx Time	integer(int64)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
```

```
        "walletType": 0,
        "walletName": "",
        "bankCard": "",
        "bankCardType": "",
        "status": 0,
        "createTime": 0,
        "createTxHash": "",
        "createTxTime": 0
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.4.2 Wallet Status Query

url: `/api/openapi/v1/token/wallet/create/result`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used to query the wallet activation result by Token ID or wallet address associated with the wallet.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoWalletCreateResultReqVO	ApiReqInfoWalletCreateResultReqVO	body	true	ApiReqInfoWalletCreateResultReqVO	ApiReqInfoWalletCreateResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	WalletCreateResultReqVO	WalletCreateResultReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress	Wallet Address		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoWalletCreateResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		WalletCreateResultRespVO	WalletCreateResultRespVO
tokenCode	Tokenized Deposit Code	string	
walletAddress	Wallet Address	string	
walletType	Wallet Type	integer(int64)	
walletName	Wallet Alias	string	
bankCard	Bank Card	string	
bankCardType	Bank Card Type	string	

name	description	type	schema
status	Status 0:Activating 1：Enable 2：Disable 3：Deactivate	integer(int32)	
createTime	Create Time	integer(int64)	
createTxHash	Create Tx Hash	string	
createTxTime	Create Tx Time	integer(int64)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "walletType": 0,
        "walletName": "",
        "bankCard": "",
        "bankCardType": "",
        "status": 0,
        "createTime": 0,
        "createTxHash": "",
        "createTxTime": 0
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.3 Get the basic wallet information

url: `/api/openapi/v1/token/wallet/detail`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by users to query wallet information by wallet address, SP ID and Token ID to which the wallet belongs.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoWalletCreateResultReqVO	ApiReqInfoWalletCreateResultReqVO	body	true	ApiReqInfoWalletCreateResultReqVO	ApiReqInfoWalletCreateResultReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	WalletCreateResultReqVO	WalletCreateResultReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress	Wallet Address		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoWalletQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	

name	description	type	schema
respTime		integer(int64)	integer(int64)
transCode		string	
body		WalletQueryRespVO	WalletQueryRespVO
tokenCode	Tokenized Deposit Code	string	
tokenName	Tokenized Deposit Name	string	
walletAddress	Wallet Address	string	
walletType	Wallet Type	integer(int64)	
walletTypeName	Wallet Type Name	string	
walletName	Wallet Alias	string	
bankCard	Bank Card	string	
bankCardType	Bank Card Type	string	
status	Status 0:Activating 1：Enable 2：Disable 3：Deactivate	integer(int32)	
createTime	Create Time	integer(int64)	
balance	Balance	number	
frozenFunds	Frozen Funds	number	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "tokenName": "",
        "walletAddress": "",
        "walletType": 0,
        "walletTypeName": "",
        "walletName": "",
        "bankCard": "",
        "bankCardType": "",
```

```
        "status": 0,
        "createTime": 0,
        "balance": 0,
        "frozenFunds": 0
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.4.4 Query wallet information by pages

url: `/api/openapi/v1/token/wallet/query`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by users to query wallet information by wallet address, SP ID and Token ID to which the wallet belongs, wallet type, creation time, and status.

Example:


```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "walletAddress": "",
      "walletType": 0,
      "startCreateTime": 0,
      "endCreateTime": 0,
```

```
      "status": 0
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableWalletQueryReqVO	ApiReqInfoDataTableWalletQueryReqVO	body	true	ApiReqInfoDataTableWalletQueryReqVO	ApiReqInfoDataTableWalletQueryReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableWalletQueryReqVO	DataTableWalletQueryReqVO
data			true	WalletQueryReqVO	WalletQueryReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress	Wallet Address		false	string	
walletType	Wallet Type		false	integer(int64)	
startCreateTime	Start Create Time		false	integer(int64)	
endCreateTime	End Create Time		false	integer(int64)	
status	State 0:Activating 1：Enable 2：Disable 3：Deactivate		false	integer(int32)	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataWalletQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataWalletQueryRespVO	ResultDataWalletQueryRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	WalletQueryRespVO
tokenCode	Tokenized Deposit Code	string	

name	description	type	schema
tokenName	Tokenized Deposit Name	string	
walletAddress	Wallet Address	string	
walletType	Wallet Type	integer(int64)	
walletTypeName	Wallet Type Name	string	
walletName	Wallet Alias	string	
bankCard	Bank Card	string	
bankCardType	Bank Card Type	string	
status	Status 0:Activating 1：Enable 2：Disable 3：Deactivate	integer(int32)	
createTime	Create Time	integer(int64)	
balance	Balance	number	
frozenFunds	Frozen Funds	number	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "tokenCode": "",
                "tokenName": "",
                "walletAddress": "",
                "walletType": 0,
                "walletTypeName": "",
                "walletName": "",
                "bankCard": "",
                "bankCardType": "",
                "status": 0,
                "createTime": 0,
                "balance": 0,
                "frozenFunds": 0
            }
        ]
    },
```

```
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.5 Query the list of my authorizations by pages

url: `/api/openapi/v1/token/wallet/queryWalletApproveToOthersList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by administrators or users to query the summary list of authorized stablecoins by the wallet address, which is the authorizer wallet that holds stablecoins.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "walletAddress": ""
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableWalletApproveReqVO	ApiReqInfoDataTableWalletApproveReqVO	body	true	ApiReqInfoDataTableWalletApproveReqVO	ApiReqInfoDataTableWalletApproveReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableWalletApproveReqVO	DataTableWalletApproveReqVO
data			true	WalletApproveReqVO	WalletApproveReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress			true	Wallet address	Wallet address
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataWalletApproveQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataWalletApproveQueryRespVO	ResultDataWalletApproveQueryRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	WalletApproveQueryRespVO
tokenCode	Tokenized Deposit Code	string	
tokenName	Tokenized Deposit Name	string	
fromWalletAddress	Authorizer Wallet Address	string	
toWalletAddress	Authorized Wallet Address	string	
approveCount	Approve Count	number	
transferCount	Transfer Count	number	
balanceCount	Balance Count	number	
mac		string	

Response Example:


```
{
    "code": "",
```

```
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "tokenCode": "",
                "tokenName": "",
                "fromWalletAddress": "",
                "toWalletAddress": "",
                "approveCount": 0,
                "transferCount": 0,
                "balanceCount": 0
            }
        ]
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.4.6 Query the list of authorization details by pages

url: `/api/openapi/v1/token/wallet/queryWalletApproveToOthersDetailList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by administrators or users to query the detailed information of each authorization by the wallet address, which is the authorizer wallet that holds stablecoins.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "fromWalletAddress": "",
      "toWalletAddress": ""
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableWalletApproveToOthersDetialReqVO	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO	body	true	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableWalletApproveToOthersDetialReqVO	DataTableWalletApproveToOthersDetialReqVO
data			true	WalletApproveToOthersDetialReqVO	WalletApproveToOthersDetialReqVO
tokenCode	Tokenized Deposit Code		true	string	
fromWalletAddress	Authorizer wallet address: the owner and holder of the stablecoins.		true	string	
toWalletAddress	Authorized wallet address: can tansfer authorizer's authorized stablecoins.		true	string	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataWalletApproveDetialQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataWalletApproveDetialQueryRespVO	ResultDataWalletApproveDetialQueryRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	WalletApproveDetialQueryRespVO
tokenName	Tokenized Deposit Name	string	
fromWalletAddress	Authorizer Wallet Address	string	
toWalletAddress	Authorized Wallet Address	string	

name	description	type	schema
approveCount	Approve Count	number	
createTime	Create Time	integer(int64)	
txHash	TxHash	string	
txTime	TxTime	integer(int64)	
status	status 20: Pending submission (pending processing) 30: Submitting (processing) 35: Submitted successfully (effective) 40: Submitted failed	integer(int32)	
type	type 1: Approve 2: Reduce Approve	integer(int32)	
transCode	transCode	string	
currentApproveCount	currentApproveCount	number	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "tokenName": "",
                "fromWalletAddress": "",
                "toWalletAddress": "",
                "approveCount": 0,
                "createTime": 0,
                "txHash": "",
                "txTime": 0,
                "status": 0,
                "type": 0,
                "transCode": "",
                "currentApproveCount": 0
            }
        ]
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.7 Query the list of authorized transfers by pages

url: `/api/openapi/v1/token/wallet/queryWalletApproveToOthersTransferList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by administrators or users to query the informaiton of transferring stablecoins by the wallet address, which is the authorized wallet that can tansfer authorizer's authorized stablecoins.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "fromWalletAddress": "",
      "toWalletAddress": ""
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableWalletApproveToOthersDetialReqVO	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO	body	true	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO	ApiReqInfoDataTableWalletApproveToOthersDetialReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableWalletApproveToOthersDetialReqVO	DataTableWalletApproveToOthersDetialReqVO
data			true	WalletApproveToOthersDetialReqVO	WalletApproveToOthersDetialReqVO
tokenCode	Tokenized Deposit Code		true	string	
fromWalletAddress	Authorizer wallet address: the owner and holder of the stablecoins.		true	string	
toWalletAddress	Authorized wallet address: can tansfer authorizer's authorized stablecoins.		true	string	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataWalletApproveTransferQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataWalletApproveTransferQueryRespVO	ResultDataWalletApproveTransferQueryRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	WalletApproveTransferQueryRespVO
tokenName	Tokenized Deposit Name	string	
fromWalletAddress	Authorizer Wallet Address	string	
senderWalletAddress	Authorized Wallet Address	string	
toWalletAddress	To Wallet Address	string	
tokenCount	tokenCount	number	
createTime	Create Time	integer(int64)	
txHash	TxHash	string	
txTime	TxTime	integer(int64)	
status	status 20: Pending submission (pending processing) 30: Submitting (processing) 35: Submitted successfully (effective) 40: Submitted failed	integer(int32)	
transCode	Trans Code	string	
fromWalletBalance	From Wallet Balance	number	
toWalletBalance	To Wallet Balance	number	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
```

```
            {
                "tokenName": "",
                "fromWalletAddress": "",
                "senderWalletAddress": "",
                "toWalletAddress": "",
                "tokenCount": 0,
                "createTime": 0,
                "txHash": "",
                "txTime": 0,
                "status": 0,
                "transCode": "",
                "fromWalletBalance": 0,
                "toWalletBalance": 0
            }
        ]
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.8 Query the list of stablecoins authorized to me by pages

url: `/api/openapi/v1/token/wallet/queryWalletApproveToMeList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used by administrators or users to query the summary list of authorized stablecoins by the wallet address, which is the authorized wallet that can tansfer authorizer's authorized stablecoins.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "tokenCode": "",
      "walletAddress": ""
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableWalletApproveReqVO	ApiReqInfoDataTableWalletApproveReqVO	body	true	ApiReqInfoDataTableWalletApproveReqVO	ApiReqInfoDataTableWalletApproveReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableWalletApproveReqVO	DataTableWalletApproveReqVO
data			true	WalletApproveReqVO	WalletApproveReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress			true	Wallet address	Wallet address
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataWalletApproveQueryRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataWalletApproveQueryRespVO	ResultDataWalletApproveQueryRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	WalletApproveQueryRespVO

name	description	type	schema
tokenCode	Tokenized Deposit Code	string	
tokenName	Tokenized Deposit Name	string	
fromWalletAddress	Authorizer Wallet Address	string	
toWalletAddress	Authorized Wallet Address	string	
approveCount	Approve Count	number	
transferCount	Transfer Count	number	
balanceCount	Balance Count	number	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "tokenCode": "",
                "tokenName": "",
                "fromWalletAddress": "",
                "toWalletAddress": "",
                "approveCount": 0,
                "transferCount": 0,
                "balanceCount": 0
            }
        ]
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.9 Query transfer record of the wallet

url: `/api/openapi/v1/token/wallet/query/transferList`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used to query wallet transfer records, which can get all transfer records with wallet addresses, and to wallet addresses as input.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "data": {
      "walletAddress": "",
      "tokenCode": "",
      "startQueryTime": 0,
      "endQueryTime": 0,
      "type": 0
    },
    "page": {
      "pageNum": 1,
      "pageSize": 10
    }
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoDataTableQueryCrossChainTransferListReqVO	ApiReqInfoDataTableQueryCrossChainTransferListReqVO	body	true	ApiReqInfoDataTableQueryCrossChainTransferListReqVO	ApiReqInfoDataTableQueryCrossChainTransferListReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	DataTableQueryCrossChainTransferListReqVO	DataTableQueryCrossChainTransferListReqVO
data			true	QueryCrossChainTransferListReqVO	QueryCrossChainTransferListReqVO
walletAddress	Wallet Address		true	string	
tokenCode	Token Code		true	string	
startQueryTime	Query start time: greater than or equal to the current time		false	integer(int64)	
endQueryTime	Query end time: Less than or equal to the current time		false	integer(int64)	
type	type 1: Normal transfer 2: Authorized transfer 3: Cross-chain transfer 4：Interest 5：Dividend		false	integer(int32)	
page			true	PageInfo	PageInfo
pageNum	pageNum		true	integer(int32)	
pageSize	pageSize		true	integer(int32)	

name	description	in	require	type	schema
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoResultDataQueryCrossChainTransferListRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ResultDataQueryCrossChainTransferListRespVO	ResultDataQueryCrossChainTransferListRespVO
page		ResultPageInfo	ResultPageInfo
total	total	integer(int64)	
rows	rows	array	QueryCrossChainTransferListRespVO
type	type 1: Normal transfer 2: Authorized transfer 3: Cross-chain transfer 4：Interest 5：Dividend	integer(int32)	
transCode	Trans Code	string	
createTime	Create Time	integer(int64)	
fromBlockchainCode	From Blockchain Code	string	
fromTokenCode	From Token Code	string	
fromTokenName	From Token Name	string	
fromTokenSymbol	From Token Symbol	string	
fromWalletAddress	Sender's Wallet Address	string	
fromTokenCount	From Token Count	number	
toBlockchainCode	To Blockchain Code	string	
toTokenCode	To Token Code	string	
toTokenName	To Token Name	string	
toTokenSymbol	To Token Symbol	string	
toWalletAddress	Receiver's Wallet Address	string	
toTokenCount	To Token Count	number	
fromStatus	From Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
fromBlockchainBrowserUrl	From Blockchain Browser Url	string	
fromTxHash	From Tx Hash	string	
fromTxTime	From Tx Time	integer(int64)	
toStatus	To Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
toBlockchainBrowserUrl	To Blockchain Browser Url	string	
toTxHash	To Tx Hash	string	

name	description	type	schema
toTxTime	To Tx Time	integer(int64)	
gasFee	Gas Fee	number	
crossChainFee	Cross Chain Fee	number	
exchangeRate	Exchange Rate	number	
status	Status 20: Pending submission 30: Submitting 35: Submission successful 40: Submission failed	integer(int32)	
transferCode	Transfer Code	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "page": {
            "total": 0
        },
        "rows": [
            {
                "type": 0,
                "transCode": "",
                "createTime": 0,
                "fromBlockchainCode": "",
                "fromTokenCode": "",
                "fromTokenName": "",
                "fromTokenSymbol": "",
                "fromWalletAddress": "",
                "fromTokenCount": 0,
                "toBlockchainCode": "",
                "toTokenCode": "",
                "toTokenName": "",
                "toTokenSymbol": "",
                "toWalletAddress": "",
                "toTokenCount": 0,
                "fromStatus": 0,
                "fromBlockchainBrowserUrl": "",
                "fromTxHash": "",
                "fromTxTime": 0,
                "toStatus": 0,
                "toBlockchainBrowserUrl": "",
                "toTxHash": "",
                "toTxTime": 0,
                "gasFee": 0,
                "crossChainFee": 0,
                "exchangeRate": 0,
                "status": 0,
                "transferCode": ""
```

```
            }
        ]
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.10 Verifiable credential changes

url: `/api/openapi/v1/token/wallet/changeVerifiableCredential`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is used to verifiable credential changes.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "verifiableCredential": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoWalletVerifiableCredentialChangeReqVO	ApiReqInfoWalletVerifiableCredentialChangeReqVO	body	true	ApiReqInfoWalletVerifiableCredentialChangeReqVO	ApiReqInfoWalletVerifiableCredentialChangeReqVO
reqTime	Request Time		true	integer(int64)	

name	description	in	require	type	schema
transCode	Request ID		true	string	
body			false	WalletVerifiableCredentialChangeReqVO	WalletVerifiableCredentialChangeReqVO
tokenCode	Tokenized Deposit Code		true	string	
walletAddress	Wallet Address		true	string	
verifiableCredential	Verifiable Credential		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoWalletVerifiableCredentialChangeResultRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		WalletVerifiableCredentialChangeResultRespVO	WalletVerifiableCredentialChangeResultRespVO
tokenCode	Tokenized Deposit Code	string	
walletAddress	Wallet Address	string	
walletType	Wallet Type	integer(int64)	
result	result	boolean	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "walletType": 0,
        "result": true
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.4.11 Verify wallet information

url: `/api/openapi/v1/token/wallet/verifyWalletInfo`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoVerifyWalletInfoReqVO	ApiReqInfoVerifyWalletInfoReqVO	body	true	ApiReqInfoVerifyWalletInfoReqVO	ApiReqInfoVerifyWalletInfoReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	VerifyWalletInfoReqVO	VerifyWalletInfoReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoVerifyWalletInfoRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		VerifyWalletInfoRespVO	VerifyWalletInfoRespVO
tokenCode	Tokenized Deposit Code	string	
walletAddress	Wallet Address	string	
walletStatus	Wallet Status 0:Activating 1：Enable 2：Disable 3：Deactivate	integer(int32)	
result	result	boolean	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "tokenCode": "",
        "walletAddress": "",
        "walletStatus": 0,
        "result": true
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5 Token Meta Transaction Module

1.5.5.1 Authorization of Increasing Amount - Create Meta Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/increase/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user increases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Increasing Amount - Create Meta Transaction" interface to compile the data into the transaction body, including the Tokenized Deposit/Stablecoin ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Increasing Amount - Send Meta Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveIncreaseMetaTransactionReqVO	ApiReqInfoCreateApproveIncreaseMetaTransactionReqVO	body	true	ApiReqInfoCreateApproveIncreaseMetaTransactionReqVO	ApiReqInfoCreateApproveIncreaseMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveIncreaseMetaTransactionReqVO	CreateApproveIncreaseMetaTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address		true	string	
toWalletAddress	Delegated Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.2 Authorization of Increasing Amount - Send Meta Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/increase/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user increases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Increasing Amount - Create Meta Transaction" interface to compile the data into the transaction body, including the Tokenized Deposit/Stablecoin ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Increasing Amount - Send Meta Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendApproveIncreaseMetaTransactionReqVO	ApiReqInfoSendApproveIncreaseMetaTransactionReqVO	body	true	ApiReqInfoSendApproveIncreaseMetaTransactionReqVO	ApiReqInfoSendApproveIncreaseMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveIncreaseMetaTransactionReqVO	SendApproveIncreaseMetaTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address		true	string	
toWalletAddress	Delegated Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message : From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveIncreaseMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveIncreaseMetaTransactionRespVO	SendApproveIncreaseMetaTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.3 Authorization of Decreasing Amount - Create Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/decrease/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user decreases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Decreasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Decreasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveDecreaseMetaTransactionReqVO	ApiReqInfoCreateApproveDecreaseMetaTransactionReqVO	body	true	ApiReqInfoCreateApproveDecreaseMetaTransactionReqVO	ApiReqInfoCreateApproveDecreaseMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveDecreaseMetaTransactionReqVO	CreateApproveDecreaseMetaTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Delegated Wallet Address: the receiver of transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	

name	description	type	schema
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.4 Authorization of Decreasing Amount - Send Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/decrease/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user decreases the amount of authorized Tokens.

Step 1: The user calls the "Authorization of Decreasing Amount - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and authorized amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorization of Decreasing Amount - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorization - Query Transaction Result" interface to query the authorization result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendApproveDecreaseMetaTransactionReqVO	ApiReqInfoSendApproveDecreaseMetaTransactionReqVO	body	true	ApiReqInfoSendApproveDecreaseMetaTransactionReqVO	ApiReqInfoSendApproveDecreaseMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveDecreaseMetaTransactionReqVO	SendApproveDecreaseMetaTransactionReqVO
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Delegated Wallet Address: the receiver of transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message: From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveDecreaseMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveDecreaseMetaTransactionRespVO	SendApproveDecreaseMetaTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.5 Authorized Transfer - Create Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers the authorized Tokens.

Step 1: The user calls the "Authorized Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, spender wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorized Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorized Transfer - Query Transaction Result" interface to query the transfer result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "spenderWalletAddress": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateApproveTransferMetaTransactionReqVO	ApiReqInfoCreateApproveTransferMetaTransactionReqVO	body	true	ApiReqInfoCreateApproveTransferMetaTransactionReqVO	ApiReqInfoCreateApproveTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateApproveTransferMetaTransactionReqVO	CreateApproveTransferMetaTransactionReqVO
spenderWalletAddress	Delegated Wallet Address: the sender with transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Receiver's Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.6 Authorized Transfer - Send Transaction

url: `/api/openapi/v1/token/metaTransaction/approve/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers the authorized Tokens.

Step 1: The user calls the "Authorized Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, sender wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculate the offline transaction hash.

Step 3: The user calls the "Authorized Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Authorized Transfer - Query Transaction Result" interface to query the transfer result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "spenderWalletAddress": "",
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema

name	description	in	require	type	schema
apiReqInfoSendApproveTransferMetaTransactionReqVO	ApiReqInfoSendApproveTransferMetaTransactionReqVO	body	true	ApiReqInfoSendApproveTransferMetaTransactionReqVO	ApiReqInfoSendApproveTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendApproveTransferMetaTransactionReqVO	SendApproveTransferMetaTransactionReqVO
spenderWalletAddress	Delegated Wallet Address: the sender with transfer authorization and able to transfer authorizer's authorized Tokens.		true	string	
fromWalletAddress	Authorizer's Wallet Address: the owner and holder of the Tokens.		true	string	
toWalletAddress	Receiver's Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message: From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendApproveTransferMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendApproveTransferMetaTransactionRespVO	SendApproveTransferMetaTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.7 Transfer - Create Transaction

url: `/api/openapi/v1/token/metaTransaction/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Transfer - Query Transaction Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateTransferMetaTransactionReqVO	ApiReqInfoCreateTransferMetaTransactionReqVO	body	true	ApiReqInfoCreateTransferMetaTransactionReqVO	ApiReqInfoCreateTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateTransferMetaTransactionReqVO	CreateTransferMetaTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	

name	description	in	require	type	schema
toWalletAddress	To Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
```

```
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.8 Transfer - Send Transaction

url: `/api/openapi/v1/token/metaTransaction/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Transfer - Create Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Transfer - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Transfer - Query Transaction Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "tokenCode": "",
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendTransferMetaTransactionReqVO	ApiReqInfoSendTransferMetaTransactionReqVO	body	true	ApiReqInfoSendTransferMetaTransactionReqVO	ApiReqInfoSendTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendTransferMetaTransactionReqVO	SendTransferMetaTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
count	Count		true	number	
tokenCode	Token Code		true	string	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message: From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendTransferMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendTransferMetaTransactionRespVO	SendTransferMetaTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.5.9 Withdrawal - Create Transaction

url: `/api/openapi/v1/token/metaTransaction/withdrawal/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user withdraws Tokens.

Step 1: The user calls the "Withdraw Token - Create Transaction" interface to compile the data into the transaction body, including the Token ID, wallet address, and withdrawal amount.

Step 2: The user uses the wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Withdraw Token - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Withdrawal Token - Query Result" interface to query the withdrawal results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "tokenCount": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateWithdrawalMetaTransactionReqVO	ApiReqInfoCreateWithdrawalMetaTransactionReqVO	body	true	ApiReqInfoCreateWithdrawalMetaTransactionReqVO	ApiReqInfoCreateWithdrawalMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateWithdrawalMetaTransactionReqVO	CreateWithdrawalMetaTransactionReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
tokenCount	Withdrawal Amount		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	
mac		string	

Response Example:


```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```


code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:


```
{
    "empty": true
}
```


1.5.5.10 Withdrawal - Send Transaction

url: `/api/openapi/v1/token/metaTransaction/withdrawal/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user withdraws Tokens.

Step 1: The user calls the "Withdraw Token - Create Transaction" interface to compile the data into the transaction body, including the Token ID, wallet address, and withdrawal amount.

Step 2: The user uses the wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Withdraw Token - Send Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Withdrawal Token - Query Result" interface to query the withdrawal results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "tokenCount": 0,
    "bankOrderNumber": "",
    "bankOrderWithdrawAccount": "",
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendWithdrawalMetaTransactionReqVO	ApiReqInfoSendWithdrawalMetaTransactionReqVO	body	true	ApiReqInfoSendWithdrawalMetaTransactionReqVO	ApiReqInfoSendWithdrawalMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendWithdrawalMetaTransactionReqVO	SendWithdrawalMetaTransactionReqVO
tokenCode	Token Code		true	string	
walletAddress	Wallet Address		true	string	
tokenCount	Withdrawal Amount		true	number	
bankOrderNumber	Bank Order Number (Length must be between 1 and 100)		true	string	
bankOrderWithdrawAccount	Bank Order Withdraw Account (Length must be between 1 and 100)		true	string	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message: From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendWithdrawalMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendWithdrawalMetaTransactionRespVO	SendWithdrawalMetaTransactionRespVO
orderNumber	Order Number	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "orderNumber": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.11 Cross Chain Transfer - Create Meta Transaction

url: `/api/openapi/v1/token/metaTransaction/crossChain/transfer/createTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Cross Chain Transfer - Create Meta Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Cross Chain Transfer - Send Meta Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Cross Chain Transfer - Query Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "fromTokenCount": 0,
    "toTokenCount": 0,
    "fromTokenCode": "",
    "toTokenCode": "",
    "exchangeRate": 0,
    "crossChainFee": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoCreateCrossChainTransferMetaTransactionReqVO	ApiReqInfoCreateCrossChainTransferMetaTransactionReqVO	body	true	ApiReqInfoCreateCrossChainTransferMetaTransactionReqVO	ApiReqInfoCreateCrossChainTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	CreateCrossChainTransferMetaTransactionReqVO	CreateCrossChainTransferMetaTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
fromTokenCount	From Token Count		true	number	
toTokenCount	To Token Count		true	number	
fromTokenCode	From Token Code		true	string	
toTokenCode	To Token Code(FromTokenCount * Exchange Rate)		true	string	
exchangeRate	Exchange Rate		true	number	
crossChainFee	Cross Chain Fee		true	number	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoCommonCreateMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		CommonCreateMetaTransactionRespVO	CommonCreateMetaTransactionRespVO
function	Function	string	
rawTransaction	Original Transaction Message	string	
gasLimit	Gas Limit	integer	
doMain	Domain	string	
expireTime	Expire Time	integer(int64)	
metaNonce	Meta Nonce	integer	
chainId	Chain Id	string	
chainName	Chain Name	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "function": "",
        "rawTransaction": "",
        "gasLimit": 0,
        "doMain": "",
        "expireTime": 0,
        "metaNonce": 0,
        "chainId": "",
        "chainName": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.5.12 Cross Chain Transfer - Send Meta Transaction

url: `/api/openapi/v1/token/metaTransaction/crossChain/transfer/sendTransaction`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

This interface is called when the user transfers Tokens.

Step 1: The user calls the "Cross Chain Transfer - Create Meta Transaction" interface to compile the data into the transaction body, including the Token ID, from wallet address, to wallet address, and transfer amount.

Step 2: The user uses the from wallet's private key to sign the transaction and calculates the offline transaction hash.

Step 3: The user calls the "Cross Chain Transfer - Send Meta Transaction" interface to upload the transaction to the blockchain.

Step 4: The user calls "Cross Chain Transfer - Query Result" interface to query the transfer results.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "fromTokenCount": 0,
    "toTokenCount": 0,
    "fromTokenCode": "",
    "toTokenCode": "",
    "exchangeRate": 0,
    "crossChainFee": 0,
    "signedMessage": "",
    "rawTransaction": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoSendCrossChainTransferMetaTransactionReqVO	ApiReqInfoSendCrossChainTransferMetaTransactionReqVO	body	true	ApiReqInfoSendCrossChainTransferMetaTransactionReqVO	ApiReqInfoSendCrossChainTransferMetaTransactionReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	SendCrossChainTransferMetaTransactionReqVO	SendCrossChainTransferMetaTransactionReqVO
fromWalletAddress	From Wallet Address		true	string	
toWalletAddress	To Wallet Address		true	string	
fromTokenCount	From Token Count		true	number	
toTokenCount	To Token Count		true	number	
fromTokenCode	From Token Code		true	string	
toTokenCode	To Token Code(FromTokenCount * Exchange Rate)		true	string	
exchangeRate	Exchange Rate		true	number	
crossChainFee	Cross Chain Fee		true	number	
signedMessage	Signed Message		true	string	
rawTransaction	Original Transaction Message : From create transaction interface response field 'rawTransaction'		true	string	
mac	Signature		true	string	

Status:

code	description	schema
200	OK	ApiRespInfoSendCrossChainTransferMetaTransactionRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		SendCrossChainTransferMetaTransactionRespVO	SendCrossChainTransferMetaTransactionRespVO
transactionId	Transaction Id	string	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "transactionId": ""
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.5.6 Common Module

1.5.6.1 Get the latest exchange rate

url: `/api/openapi/v1/common/fx/get`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

By calling this interface, you can get the latest exchange rate of the specified currency.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "sendCurrencySymbol": "",
    "receiveCurrencySymbol": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoExchangeRateReqVO	ApiReqInfoExchangeRateReqVO	body	true	ApiReqInfoExchangeRateReqVO	ApiReqInfoExchangeRateReqVO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			false	ExchangeRateReqVO	ExchangeRateReqVO
sendCurrencySymbol	Receive Token Pegged Currency Symbol		true	string	
receiveCurrencySymbol	Send Token Pegged Currency Symbol		true	string	
mac	Signature		true	string	

Status:

code	description	schema

code	description	schema
200	OK	ApiRespInfoExchangeRateRespVO
405	Method Not Allowed	R

code-200:

Response Params:

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		ExchangeRateRespVO	ExchangeRateRespVO
sendCurrencySymbol	Send Currency Symbol	string	
receiveCurrencySymbol	Receive Currency Symbol	string	
exchangeRate	Exchange Rate	number	
updateTime	Update Time (The timestamp (13-digit decimal, millisecond precision))	integer(int64)	
mac		string	

Response Example:

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": {
        "sendCurrencySymbol": "",
        "receiveCurrencySymbol": "",
        "exchangeRate": 0,
        "updateTime": 0
    },
    "mac": ""
}
```

code-405:

Response Params:

name	description	type	schema
empty		boolean	

Response Example:

```
{
    "empty": true
}
```

1.6 Webhooks

1.6.1 Webhooks Calling Mechanism

After the Service Provider invokes OpenAPIs (such as wallet activation, purchase, withdrawal, transfer, cross-chain transfer, and authorization-related interfaces), the System processes on-chain transactions asynchronously and sends callback notifications to the Service Provider's configured Webhooks URL.

1.6.1.1 Common Callback Headers

1. Configure the HTTP request header to include `key = accessKey`, where `value` is the access key provided by the Tokenized Deposit/Stablecoin Management System.
2. Configure the HTTP request header to include `key = Content-Type`, where `value` is `application/json`.

1.6.1.2 Callback Request Format (`ApiReqInfo<T>`)

Parameter name	Description	Data type	Required
transCode	Unique code of the request	String	true
reqTime	The timestamp (13-digit decimal, millisecond precision) at which the System sends the callback request	Long	true
body	Request body (as defined by the specific business)	Object	true
mac	Signature	String	true

1.6.1.3 Common Callback Request Example

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {},
  "mac": ""
}
```

1.6.1.4 Callback Handling Requirement

The Service Provider receives the callback, verifies the signature using the gateway public key, and returns a standard success response.

1.6.1.5 Callback Response Format (ApiRespInfo)

name	description	type	schema
code		string	
respTime		integer(int64)	integer(int64)
transCode		string	
body		Boolean	Boolean
mac		string	

1.6.1.6 Callback Response Example

```
{
    "code": "",
    "respTime": 0,
    "transCode": "",
    "body": true,
    "mac": ""
}
```

1.6.2 Wallet Create Result Notify

url: `/v1/token/wallet/create/result/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for wallet activation result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "walletType": 0,
    "walletName": "",
    "bankCard": "",
```

```
    "bankCardType": "",
    "status": 0,
    "createTime": 0,
    "createTxHash": "",
    "createTxTime": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoOpenWalletDTO	ApiReqInfoOpenWalletDTO	body	true	ApiReqInfoOpenWalletDTO	ApiReqInfoOpenWalletDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	OpenWalletDTO	OpenWalletDTO
tokenCode	Token Code		false	string	
walletAddress	Wallet Address		false	string	
walletType	Wallet Type		false	integer(int64)	
walletName	Wallet Alias		false	string	
bankCard	Bank Card		false	string	
bankCardType	Bank Card Type		false	string	
status	Status 1: Enable 2: Disable		false	integer(int32)	
createTime	Create Time		false	integer(int64)	
createTxHash	Create Tx Hash		false	string	
createTxTime	Create Tx Time		false	integer(int64)	
mac	Signature		true	string	

1.6.3 Token Buy Result Notify

url: `/v1/token/manage/buy/result/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for Token purchase result.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
    "walletAddress": "",
    "bankOrderNumber": "",
    "bankOrderPayTime": 0,
```

```
    "bankOrderPayAccount": "",
    "tokenCount": 0,
    "orderNumber": "",
    "status": 0,
    "txHash": "",
    "txTime": 0,
    "transCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenBuyDTO	ApiReqInfoTokenBuyDTO	body	true	ApiReqInfoTokenBuyDTO	ApiReqInfoTokenBuyDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	TokenBuyDTO	TokenBuyDTO
tokenCode	Token Code		false	string	
walletAddress	Wallet Address		false	string	
bankOrderNumber	Bank Order Number		false	string	
bankOrderPayTime	Bank Order Pay Time		false	integer(int64)	
bankOrderPayAccount	Bank Order Pay Account		false	string	
tokenCount	Amount		false	number	
orderNumber	Order Number		false	string	
status	Status 35: Submission successful 40: Submission failed		false	integer(int32)	
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
transCode	Request Trans Code		false	string	
mac	Signature		true	string	

1.6.4 Token Withdrawal Result Notify

url: `/v1/token/manage/withdrawal/result/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for Token withdrawal and meta-transaction token withdrawal result.

Example:Z

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "tokenCode": "",
```

```
    "walletAddress": "",
    "tokenCount": 0,
    "orderNumber": "",
    "bankOrderNumber": "",
    "bankOrderWithdrawAccount": "",
    "status": 0,
    "txHash": "",
    "txTime": 0,
    "transCode": ""
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTokenWithdrawalDTO	ApiReqInfoTokenWithdrawalDTO	body	true	ApiReqInfoTokenWithdrawalDTO	ApiReqInfoTokenWithdrawalDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	TokenWithdrawalDTO	TokenWithdrawalDTO
tokenCode	Token Code		false	string	
walletAddress	Wallet Address		false	string	
tokenCount	Amount		false	number	
orderNumber	Order Number		false	string	
bankOrderNumber	Bank Order Number		false	string	
bankOrderWithdrawAccount	Bank Order Withdraw Account		false	string	
status	Status 35: Submission successful 40: Submission failed		false	integer(int32)	
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
transCode	Request Trans Code		false	string	
mac	Signature		true	string	

1.6.5 Transfer Result Notify

url: `/v1/token/transaction/transfer/queryResult/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for transfer and meta-transaction transfer result query.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "txHash": "",
```

```
    "txTime": 0,
    "fromWalletAddress": "",
    "toWalletAddress": "",
    "count": 0,
    "transCode": "",
    "status": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTransferResultDTO	ApiReqInfoTransferResultDTO	body	true	ApiReqInfoTransferResultDTO	ApiReqInfoTransferResultDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	TransferResultDTO	TransferResultDTO
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
fromWalletAddress	Authorizer's Wallet Address		false	string	
toWalletAddress	Receiver's Wallet Address		false	string	
count	Count		false	number	
transCode	Trans Code		false	string	
status	Status 35: Submitted 40: Submission failed		false	integer(int32)	
mac	Signature		true	string	

1.6.6 Authorized Result Notify

url: `/v1/token/transaction/approve/queryResult/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for authorized and meta-transaction authorized related result query.

Example:

```
{
  "reqTime": 0,
     "transCode": "",
     "body": {
        "transCode": "",
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "count": 0,
        "txHash": "",
        "txTime": 0,
        "status": 35
```

```
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoApproveTransactionResultDTO	ApiReqInfoApproveTransactionResultDTO	body	true	ApiReqInfoApproveTransactionResultDTO	ApiReqInfoApproveTransactionResultDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	ApproveTransactionResultDTO	ApproveTransactionResultDTO
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
fromWalletAddress	Authorizer's Wallet Address		false	string	
toWalletAddress	Receiver's Wallet Address		false	string	
count	Count		false	number	
transCode	Trans Code		false	string	
status	Status 35: Submitted 40: Submission failed		false	integer(int32)	
mac	Signature		true	string	

1.6.7 Authorized Transfer Result Notify

url: `/v1/token/transaction/approve/transfer/queryResult/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for authorized transfer and meta-transaction authorized transfer related result query.

Example:

```
{
  "reqTime": 0,
      "transCode": "",
      "body": {
        "txHash": "",
        "txTime": 0,
        "spenderWalletAddress": "",
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "count": 0,
        "transCode": "",
        "status": 0
  },
  "mac": ""
}
```

Params:

name	description	in	require	type	schema
apiReqInfoTransferResultDTO	ApiReqInfoTransferResultDTO	body	true	ApiReqInfoTransferResultDTO	ApiReqInfoTransferResultDTO
reqTime	Request Time		true	integer(int64)	

name	description	in	require	type	schema
transCode	Request ID		true	string	
body			true	TransferResultDTO	TransferResultDTO
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
spenderWalletAddress	Delegated Wallet Address		false	string	
fromWalletAddress	Authorizer's Wallet Address		false	string	
toWalletAddress	Receiver's Wallet Address		false	string	
count	Count		false	number	
transCode	Trans Code		false	string	
status	Status 35: Submitted 40: Submission failed		false	integer(int32)	
mac	Signature		true	string	

1.6.8 Cross-Chain Transfer Result Notify

url: `/v1/token/transaction/crossChain/transfer/queryResult/notify`

method: `POST`

produces: `application/x-www-form-urlencoded,application/json`

consumes: `*/*`

Note:

Callback for cross-chain transfer and meta-transaction cross-chain transfer result query.

Example:

```
{
  "reqTime": 0,
  "transCode": "",
  "body": {
    "txHash": "",
    "txTime": 0,
    "fromWalletAddress": "",
    "fromTokenCode": "",
    "fromTokenName": "",
    "fromTokenSymbol": "",
    "toWalletAddress": "",
    "toTokenCode": "",
    "toTokenName": "",
    "toTokenSymbol": "",
    "fromTokenCount": 0,
    "crossChainFee": 0,
    "toTokenCount": 0,
    "transactionId": "",
    "status": 0,
    "exchangeRate": 0,
    "transferCode": "",
    "detailList": [
      {
        "tokenCode": "",
```

```
        "tokenName": "",
        "tokenSymbol": "",
        "tokenCurrencySymbol": "",
        "blockCode": "",
        "blockName": "",
        "blockShortName": "",
        "detailType": 0,
        "txHash": "",
        "txTime": 0,
        "txStatus": 0,
        "senderWalletAddress": "",
        "fromWalletAddress": "",
        "toWalletAddress": "",
        "tokenCount": 0,
        "createTime": 0,
        "txChainType": 0,
        "handlerCode": "",
        "remarks": ""
      }
    ]
  },
  "mac": ""
}
```


Params:

name	description	in	require	type	schema
apiReqInfoCrossChainTransactionResultDTO	ApiReqInfoCrossChainTransactionResultDTO	body	true	ApiReqInfoCrossChainTransactionResultDTO	ApiReqInfoCrossChainTransactionResultDTO
reqTime	Request Time		true	integer(int64)	
transCode	Request ID		true	string	
body			true	CrossChainTransactionResultDTO	CrossChainTransactionResultDTO
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
fromWalletAddress	From Wallet Address		false	string	
fromTokenCode	From Token Code		false	string	
fromTokenName	From Token Name		false	string	
fromTokenSymbol	From Token Symbol		false	string	
toWalletAddress	To Wallet Address		false	string	
toTokenCode	To Token Code		false	string	
toTokenName	To Token Name		false	string	
toTokenSymbol	To Token Symbol		false	string	
fromTokenCount	From Token Count		false	number	
crossChainFee	Cross Chain Fee		false	number	
toTokenCount	To Token Count		false	number	
transactionId	Transaction Id		false	string	
status	Status 35: Submission successful 40: Submission failed		false	integer(int32)	
exchangeRate	Exchange Rate		false	number	
transferCode	Transfer Code		false	string	
detailList	Cross Chain Transaction Detail List		false	array	CrossChainTransactionsDetailsRespVO
tokenCode	Tokenized Deposit Code		false	string	
tokenName	Token Name		false	string	
tokenSymbol	Token Symbol		false	string	
tokenCurrencySymbol	Token Currency Symbol		false	string	
blockCode	Block Code		false	string	
blockName	Block Name		false	string	
blockShortName	Block Short Name		false	string	
detailType	Detail Type 1: Source chain initiates transfer 5: Source chain verifies transaction 10: Verify exchange rate 15: Submit verification result 20: Target chain initiates transfer 25: Source chain initiates refund		false	integer(int32)	
txHash	Tx Hash		false	string	
txTime	Tx Time		false	integer(int64)	
txStatus	Tx Status 35: Submission successful 40: Submission failed		false	integer(int32)	
senderWalletAddress	Sender Wallet Address		false	string	
fromWalletAddress	From Wallet Address		false	string	

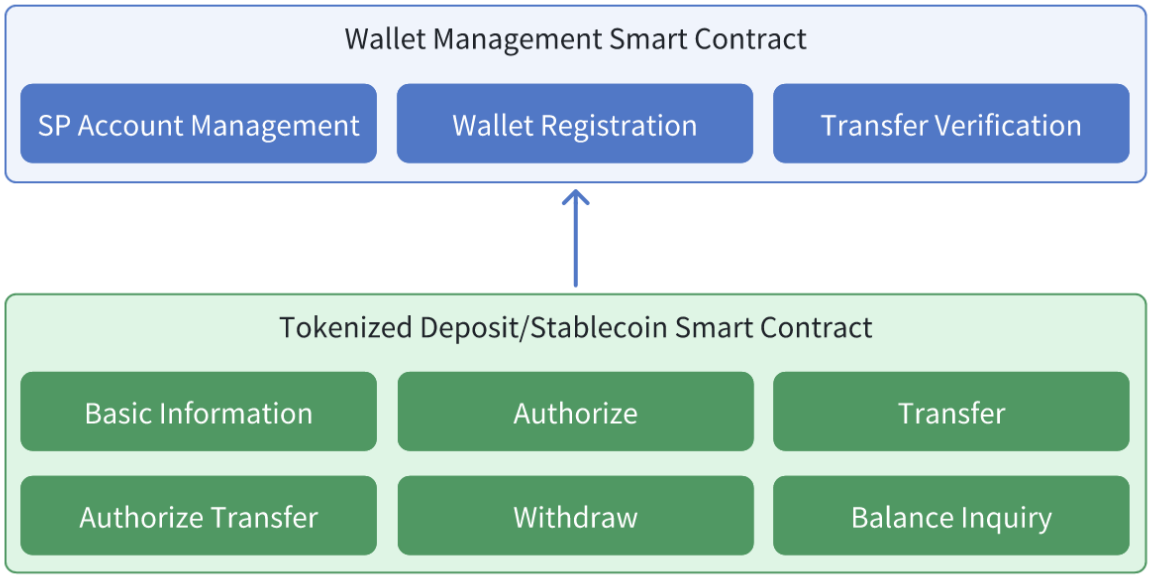
name	description	in	require	type	schema
toWalletAddress	To Wallet Address		false	string	
tokenCount	Token Count		false	number	
createTime	Create Time		false	integer(int64)	
txChainType	Tx Chain Type 0: Exchange rate service 1: Source chain 2: Target chain		false	integer(int32)	
handlerCode	Handler Code		false	string	
remarks	Remarks		false	string	
mac	Signature		true	string	

2. Access by Calling the Smart Contract or the Node

2.1 Solidity-based Smart Contract

```
solidity 0.8.9+;
openzeppelin 4.8.3;
```

2.1.1 Introduction



2.1.2 Contract Interfaces

2.1.2.1 Token Contract Interfaces

Interface Files

```
 SPDX-License-Identifier: MIT
pragma solidity ^0.8.9;

interface IStablecoin {
    /**
     * @dev Emitted when `value` stablecoins are moved from one wallet (`from`) to
     * another (`to`).
     *
     * Note that `value` can be zero.
```

```
     */
    event Transfer(address indexed from, address indexed to, uint256 value);

    event TransferFrom(
        address indexed sender,
        address indexed from,
        address indexed to,
        uint256 value
    );

    event Balance(address indexed account, uint256 balance);

    event WithdrawalCash(
        address indexed sender,
        address indexed owner,
        uint256 amount
    );

    event SetRepositoryAddress(
        address indexed operator,
        address newAddr,
        address oldAddr
    );
    event SetWalletAddress(
        address indexed operator,
        address newAddr,
        address oldAddr
    );
    /**
     * @dev Emitted when a `spender` is authorized by an `owner` by
     * calling {approve}. `value` is the new authorization.
     */
    event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );

    event ApprovalChange(
        address indexed owner,
        address indexed spender,
        uint256 value,
        bool increase,
        uint256 amount
    );

    function mintStablecoin(uint256 amount) external returns (bool);

    function burnStablecoin(uint256 amount) external returns (bool);

    function setExchange(string memory currency, uint256 rate) external;
```

```
    /**
     * @dev Returns the number of stablecoins in existence.
     */
    function totalSupply() external view returns (uint256);

    /**
     * @dev Returns the number of stablecoins owned by `account`.
     */
    function balanceOf(address account) external returns (uint256);

    /**
     * @dev Moves `amount` stablecoins from the caller's wallet to `to` wallet.
     *
     * Returns a boolean value indicating whether the operation succeeded.
     *
     * Emits a {Transfer} event.
     */
    function transfer(address to, uint256 amount) external returns (bool);

    /**
     * @dev Returns the remaining number of stablecoins that `spender` is
     * allowed to spend on behalf of `owner` through {transferFrom}. This
     * value is zero by default.
     *
     * This value changes when {approve} or {transferFrom} are called.
     */
    function allowance(
        address owner,
        address spender
    ) external view returns (uint256);

    /**
     * @dev Set `amount` stablecoins that can be tranferred by `spender` authorized
by the caller.
     *
     * Returns a boolean value indicating whether the operation succeeded or not.
     *
     * IMPORTANT: Beware that changing an authorization with this function brings
the risk
     * that someone may use both the old and the new authorization by a wrong
     * transaction order. One possible solution to mitigate this race
     * condition is to first reduce the spender's authorized stablecoins to zero and
set the
     * desired value afterwards:
     * https://github.com/ethereum/EIPs/issues/20#issuecomment-263524729
     *
     * Emits an {Approval} event.
     */
    function approve(address spender, uint256 amount) external returns (bool);

    function approveIncrease(
        address spender,
```

```
        uint256 amount
    ) external returns (bool);

    function approveDecrease(
        address spender,
        uint256 amount
    ) external returns (bool);

    /**
     * @dev Moves `amount` stablecoins from `from` wallet to `to` wallet using the
     * authorization mechanism. `amount` is then deducted from the caller's
     * authorization amount.
     *
     * Returns a boolean value indicating whether the operation was successful.
     *
     * Emits a {Transfer} event.
     */
    function transferFrom(
        address from,
        address to,
        uint256 amount
    ) external returns (bool);

     /**
     * @dev The user calls this function to transfer `amount` stablecoins back to
the repository wallet.
     *
     * Emits a {Transfer} event.
     */
    function withdrawByUser(uint256 amount) external returns (bool);
}
```

Transfer

Method

```
function transfer(address to, uint256 amount) external returns (bool);
```

Description

```
to: the receiver's wallet address
amount: the amount to be transferred out
```

Event

```
event Transfer(address indexed from, address indexed to, uint256 value);
event Balance(address indexed account, uint256 balance)
```

Authorized Transfer

Method

```
function transferFrom( address from, address to, uint256 amount) external returns
(bool);
```

Description

```
from: the sender's wallet address
to: the receiver's wallet address
amount: the amount's to be transferred
```

Event

```
event Transfer(address indexed from, address indexed to, uint256 value);
event TransferFrom(address indexed sender,address indexed from, address indexed to,
uint256 value)
event Balance(address indexed account, uint256 balance)
```

Authorization

Method

```
function approve(address spender, uint256 amount) external returns (bool);
```

Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the authorized amount
```

Event

```
event Approval(address indexed owner, address indexed spender, uint256 value);
```

Increase Authorization

Method

```
function approveIncrease(address spender, uint256 amount) public returns (bool);
```

Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the amount of increased authorization
```

Event

```
event ApprovalChange(
        address indexed owner,
        address indexed spender,
        uint256 value,
        bool increase, //true
        uint256 amount
    );

event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );
```

Decrease Authorization

Method

```
function approveDecrease(address spender, uint256 amount) public returns (bool)
```

Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the amount of decreased authorization
```

Event

```
event ApprovalChange(
        address indexed owner,
        address indexed spender,
        uint256 value,
        bool increase, //false
        uint256 amount //new amount
    );
     event Approval(
        address indexed owner,
        address indexed spender,
        uint256 value
    );
```

Authorization Query

Method

```
function allowance(address owner, address spender) external view returns (uint256);
```

Description

```
owner: the authorizer's wallet address
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Tokens
Returns the remaining amount of authorized transferrable Tokens
```

Balance Query

Method

```
function balanceOf(address account) external view returns (uint256);
```

Description

```
account: the wallet address to be queried
Returns the balance of the wallet
```

User Withdrawal

Method

```
function withdrawByUser(uint256 amount) external returns (bool);
```

Description

```
amount: the withdrawal amount
Return whether the result is successful
```

2.1.2.2 Token Wallet Contract Interfaces

Interface Files

```
 SPDX-License-Identifier: MIT
pragma solidity ^0.8.9;

interface IWalletManage {
    /**
     * @dev Current status of a Wallet type.
     */
```

```
    enum WalletTypeStatus {
        Init,
        Enabled,
        Disabled
    }

    /**
     * @dev Current status of a wallet.
     */
    enum walletStatus {
        Init,
        Enabled,
        Disabled
    }

    struct WalletTypeInfo {
        uint256 walletTypeId;
        string name;
        WalletTypeStatus status;
        uint256 depositLimit;
        uint256 dailyTransLimit;
        uint256 perTransLimit;
    }

    struct SPInfo {
        uint256 id;
        string code;
        bool status;
    }

    /**
     * @dev The limit and status information of a wallet.
     * @param isRegist: true:registered; false:unregistered or canceled.
     * @param isUseDefault: true:uses the wallet type default limit; false:uses a
separate limit.
     * @param depositLimit: deposit amount limit.
     * @param dailyTransLimit: daily transfer amount limit.
     * @param perTransLimit: maximum transfer amount per transaction.
     * @param freezeBalance: frozen balance.
     * @param creator: registrant's wallet address.
     */
    struct walletInfo {
        uint256 walletTypeId;
        bool isRegist;
        walletStatus status;
        bool isUseDefault;
        uint256 depositLimit;
        uint256 dailyTransLimit;
        uint256 perTransLimit;
        uint256 freezeBalance;
        address creator;
        uint256 spID;
```

```
    }

    /**
     * @dev The wallet transfer amount for the day.
     * @param date: Identification of the date.
     * @param currentTransAmount: Transfer amount of the day.
     */
    struct walletTransInfo {
        bytes32 date;
        uint256 currentTransAmount;
    }

    /**
     * @dev Represents the wallet limit information.
     */
    struct walletLimitInfo {
        uint256 depositLimit;
        uint256 dailyTransLimit;
        uint256 perTransLimit;
        uint256 freezeBalance;
    }

    event RegisterServiceProviderOrganisation(
        address indexed sender,
        string code,
        uint256 id,
        uint256[] walletTypes,
        address[] accounts
    );

    event UpdateServiceProviderOrganisationWalletTypes(
        address indexed sender,
        string code,
        uint256 id,
        uint256[] walletTypes,
        bool[] walletAuths
    );

    event UpdateServiceProviderOrganisationStatus(
        address indexed sender,
        string code,
        uint256 id,
        bool status
    );

    event SetServiceProviderAccount(
        address indexed sender,
        string code,
        uint256 id,
        address[] accounts,
        bool[] accountsAuth
    );
```

```
    /**
     * @dev Emitted when set or update RepositoryManage proxy contract address.
     */
    event SetRepositoryManageAddr(
        address indexed operator,
        address newAddr,
        address oldAddr
    );

    /**
     * @dev Emitted when set or update stablecoin proxy contract address.
     */
    event SetStablecoinAddr(
        address indexed operator,
        address newAddr,
        address oldAddr
    );

    /**
     * @dev Emitted when set or add bank address.
     */
    event SetBankAddr(address indexed operator, address indexed bankAddr);
    event SetOperationAddr(
        address indexed operator,
        address indexed operationAddr
    );

    /**
     * @dev Emitted when delete bank address.
     */
    event DelBankAddr(address indexed operator, address indexed bankAddr);
    event DelOperationAddr(
        address indexed operator,
        address indexed operationAddr
    );

    /**
     * @dev Emitted when set or update default limit info of a Wallet type.
     */
    event SetWalletTypeInfo(
        address indexed operator,
        uint256 walletTypeId,
        string name,
        WalletTypeStatus status,
        uint256 depositLimit,
        uint256 dailyTransLimit,
        uint256 perTransLimit
    );

    event SetWalletTypeStatus(
        address indexed operator,
```

```
        uint256 walletTypeId,
        WalletTypeStatus status
    );

    /**
     * @dev Emitted when a wallet is being registered.
     */
    event RegisterWallet(
        address indexed operator,
        address indexed walletAddress,
        uint256 walletTypeId,
        string code,
        bytes data
    );

    event RegisterWalletResult(
        address indexed operator,
        address indexed walletAddress,
        bool result,
        string errCode
    );

    event UpdateWalletType(
        address indexed operator,
        address indexed walletAddress,
        uint256 walletTypeId
    );

    /**
     * @dev Emitted when a wallet status is updated.
     */
    event SetWalletStatus(
        address indexed operator,
        address indexed walletAddress,
        walletStatus status,
        bytes data
    );

    /**
     * @dev Emitted when a wallet is freezing or unfreezing the wallet balance.
     */
    event SetWalletControlAmount(
        address indexed operator,
        address indexed walletAddress,
        uint256 amount,
        bytes data
    );

    /**
     * @dev Emitted when a wallet is canceled.
     */
    event CancelWallet(
```

```
        address indexed operator,
        address indexed walletAddress,
        bytes data
    );

    /**
     * @dev Emitted when setting the limit of a particular wallet.
     */
    event SetWalletLimit(
        address indexed operator,
        address indexed walletAddress,
        uint256 depositLimit,
        uint256 dailyTransLimit,
        uint256 perTransLimit
    );

    /**
     * @dev Emitted when cancelling the limit of a particular wallet.
     */
    event CencelWalletLimit(
        address indexed operator,
        address indexed walletAddress
    );

    event SetRegisterStatus(address indexed operator, bool status);
    event SetTDStatus(address indexed operator, bool status);
    event UpdateServiceProviderCode(
        address indexed sender,
        string oldCode,
        string newCode
    );
    /**
     * @dev Get the walletTypeId type of the wallet the ServiceProvider is
authorized to register..
     */
    function getServiceProviderWalletTypes(
        string memory code
    ) external view returns (uint256[] memory);

    /**
     * @dev Get all available bank address.
     */
    function getBankAddr() external view returns (address[] memory);

    /**
     * @dev Get all available Operation address.
     */
    function getOperationAddr() external view returns (address[] memory);

    /**
     * @dev Get the information of a wallet type.
     */
```

```
    function getWalletTypeInfo(
        uint256 walletTypeId
    ) external view returns (WalletTypeInfo memory);

    /**
     * @dev Register wallet wallet type and wallet info.
     * Requirements:
     * - the sender must be the ServiceProvider address.
     */
    function registerWalletBySP(
        address walletAddress,
        uint256 walletTypeId,
        bytes memory data
    ) external;

    /**
     * @dev Get the account of ServiceProvider code.
     */
    function getAccountSPCode(
        address account
    ) external view returns (string memory);

    /**
     * @dev Get a wallet limit info.
     */
    function getWalletLimit(
        address walletAddress
    ) external view returns (walletLimitInfo memory);

    /**
     * @dev Get the number of stablecoins of a wallet that can be transferred out
for the day.
     */
    function getWalletSurplusLimit(
        address walletAddress
    ) external view returns (uint256);

    /**
     * @dev Get the status of a wallet.
     */
    function getWalletStatus(
        address walletAddress
    ) external view returns (walletStatus);

    /**
     * @dev Get the wallet type of a wallet.
     */
    function getWalletType(
        address walletAddress
    ) external view returns (uint256);

    /**
```

```
     * @dev Get the number of stablecoins already tranferred for the day.
     */
    function getTransferAmount(
        address walletAddress
    ) external returns (uint256);

    /**
     * @dev Get ServiceProvider ID by ServiceProvider code.
     */
    function getSPID(string memory code) external view returns (uint256);

    /**
     * @dev Get ServiceProvider code by ServiceProvider ID.
     */
    function getSPCode(uint256 id) external view returns (string memory);

    /**
     * @dev Get the ServiceProvider ID of the account.
     */
    function getAccountOfSP(address addr) external view returns (uint256);

    /**
     * @dev Get the ServiceProvider ID of the wallet.
     */
    function getWalletOfSP(address addr) external view returns (uint256);
}
```

Register a Wallet

Method

```
function registerWalletBySP(
        address walletAddress,
        uint256 walletTypeId,
        bytes memory data
    ) external;
```

Description

```
walletAddress: the wallet address to be registered
walletTypeId: the type of wallet
data: remarks
```

Event

```
event RegisterWallet(
        address indexed operator,
        address indexed walletAddress,
        uint256 walletTypeId,
        string code,
        bytes data
    );
```

Get the List of Service Provider Wallet Type

Method

```
    function getServiceProviderWalletTypes(
        string memory code
    ) external view returns (uint256[] memory);
```

Description

```
code: Service Provider code
Returns a list of authorized wallet types
```

Get Wallet Type Information

Method

```
    function getWalletTypeInfo(uint256 walletTypeId) external view returns
(WalletTypeInfo memory);
```

Description

```
walletTypeId: the wallet type ID

Return wallet type information:
walletTypeId: the wallet type ID
name: the name of the wallet type
status: 0: Initialized 1: Enabled 2: Disabled
depositLimit: the maximum balance limit
dailyTransLimit: the maximum daily transfer-out limit
perTransLimit: the limit for each transfer-out
```

Get the Service Provider Code by Account

Method

```
    function getAccountSPCode(
        address account
    ) external view returns (string memory);
```

Description

```
account: the Service Provider account address
Returns the Service Provider code
```

Get Wallet Limit Information

Method

```
    function getWalletLimit(
        address walletAddress
    ) external view returns (walletLimitInfo memory);
```

Description

```
walletAddress: the wallet address

Return wallet limit information
depositLimit: the maximum balance limit
dailyTransLimit: the daily transfer-out limit
perTransLimit: the limit for each transfer-out
freezeBalance: the amount that is frozen
```

Get Transferable Token Balance

Method

```
    function getWalletSurplusLimit(
        address walletAddress
    ) external view returns (uint256);
```

Description

```
walletAddress: the wallet address
Return the transferable Token balance of the wallet on the day
```

Get Wallet Status

Method

```
    function getWalletStatus(
        address walletAddress
    ) external view returns (walletStatus);
```

Description

```
walletAddress: the wallet address
Return the wallet status
0: Not registered
1: Enabled
2: Disabled
```

Get the Wallet Type ID

Method

```
    function getWalletType(
        address walletAddress
    ) external view returns (uint256);
```

Description

```
walletAddress: the wallet address
Returns the wallet type ID
```

Retrieve the Daily Transferred-Out Amount

Method

```
    function getTransferAmount(
        address walletAddress
    ) external returns (uint256);
```

Description

```
walletAddress: the wallet address
Return the amount that has been transferred out on the day
```

Get the Service Provider ID by the Service Provider Code

Method

```
    function getSPID(string memory code) external view returns (uint256);
```

Description

```
code: Service Provider code
Return the Service Provider ID
```

Get the Service Provider Code by the Service Provider ID

Method

```
    function getSPCode(uint256 id) external view returns (string memory);
```

Description

```
ID: Service Provider ID
Returns the Service Provider code
```

Get the Service Provider ID by Account

Method

```
    function getAccountOfSP(address addr) external view returns (uint256);
```

Description

```
addr: the account address
Return the ServiceProvider ID to which the service belongs
```

Get the ServiceProvider ID by Wallet

Method

```
    function getWalletOfSP(address addr) external view returns (uint256);
```

Description

```
addr: the wallet address
Returns the Service Provider ID to which the service belongs
```

2.1.3 Contract Interface Status Code

Exception Code	Description	Remarks
E0001	No permission	The caller does not have permission to call the method
E0002	The address cannot be empty	Wallet address/contract address cannot be empty
E0003	Invalid operation or no operation required. For example, when you edit a parameter, this code will be returned if the new value is the same as the original one.	
E0004	The amount of Tokens must be greater than 0	
E1001	The account address already exists	Used for the verification of adding the address of a bank account or operation account
E1002	The account address does not exist	Used for the verification of deleting the address of a bank account or operation account
E1003	Wrong wallet type	Verify the wallet type
E1004	Incorrect wallet status type	Verify the wallet status type
E1005	The wallet has been disabled	
E1006	The wallet has been cancelled or is not registered yet	
E1007	The from address and to address cannot be the same	
E1008	Single transfer exceeds the limit	
E1009	The transfer limit on the day is insufficient or has exceeded	
E1010	Wallet type is not available	
E1011	The Service Provider organization name has been registered	You need to change the Service Provider name

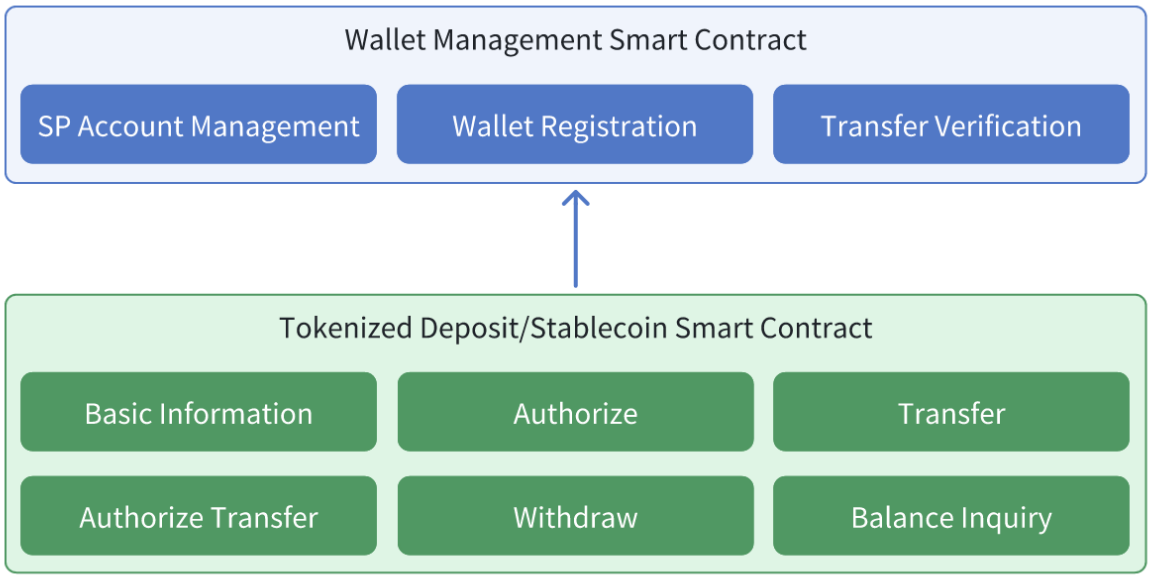
Exception Code	Description	Remarks
E1012	The Service Provider organization is not registered	You need to register the Service Provider name first, or change the Service Provider name to a registered Service Provider name
E1013	The number of array type parameters does not match or is 0	Check the parameters, the number of parameters in the two lists does not match
E1014	This account has already been registered and used by another Service Provider	
E1015	Sender does not have permission to register the wallet	
E1016	When using a contract to call the registered wallet method, the contract address and the signer's address are not the same Service Provider organization	
E1017	The wallet type is not authorized to register the wallet for this Service Provider organization	
E1018	The Service Provider organization has been disabled	
E1019	When decreasing the authorization amount, the decreased value is greater than the original one	
E1020	Service Provider registration is disabled	Contact your bank to activate Service Provider registration
E1021	The wallet has already been registered, and it cannot be registered again	
E1022	Account balance reaches the limit	You need to reduce the transfer amount
E2001	Insufficient account balance	Insufficient balance in the transfer account
E2002	Insufficient authorization amount	The authorized amount when authorizing a transfer is less than the transfer amount

Exception Code	Description	Remarks
E2003	Transfer verification failed	Failed to verify the transfer amount
E2004	Insufficient available balance	The available amount that can be transferred-out in the account is insufficient
E2005	Exceeds account limit	The transfer of funds exceeds the total account limit
E3001	The exchange rate needs to be greater than 0	
E3002	Transfer failed	

2.2 Solidity-based Smart Contract

```
aptos 4.2.3;
```

2.2.1 Introduction



2.2.2 Contract Interfaces

2.2.2.1 Token Contract Interfaces

Interface Files

```
module td_stablecoin::stablecoin {
    use td_stablecoin::wallet_manage;
    friend td_stablecoin::repository_manage;

    use std::option;
    use std::signer;
    use std::string::{utf8, String};
    use std::simple_map::{Self, SimpleMap};

    use aptos_framework::fungible_asset::{Self, MintRef, TransferRef, BurnRef,
Metadata,MutateMetadataRef};
    use aptos_framework::object::{Self, Object, ExtendRef};
    use aptos_framework::primary_fungible_store;
    use aptos_framework::account;
    use aptos_framework::event;
    use aptos_framework::chain_id;

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct StablecoinMetadata has key {
        has_repository: bool,
        exchange: SimpleMap<String, u64>,
        allowances: SimpleMap<address, SimpleMap<address, u64>>,
        event_signer_cap: account::SignerCapability,
        metadata_object_address:address,
    }

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct Management has key {
        extend_ref: ExtendRef,
        mint_ref: MintRef,
        burn_ref: BurnRef,
        transfer_ref: TransferRef,
        mutate_metadata_ref: MutateMetadataRef,
    }

    struct StablecoinEventHandle has key {
        mint_events : event::EventHandle<Mint>,
        burn_events : event::EventHandle<Burn>,
        transfer_events : event::EventHandle<Transfer>,
        transfer_from_events : event::EventHandle<TransferFrom>,
        balance_events : event::EventHandle<Balance>,
        approval_events : event::EventHandle<Approval>,
        withdrawal_cash_events : event::EventHandle<WithdrawalCash>,
        approval_change_events : event::EventHandle<ApprovalChange>,
        set_stablecoin_info_events : event::EventHandle<SetStablecoinInfo>,
    }
```

```
struct ApprovalMsg has drop {
    owner: address,
    to: address,
    nonce: u64,
    chain_id: u8,
    spender: address,
    amount: u64
}

#[event]
struct SetStablecoinInfo has drop, store {
    sender: address,
    name: String,
    symbol: String,
    decimals: u8
}

#[event]
struct Mint has drop, store {
    minter: address,
    to: address,
    amount: u64
}

#[event]
struct Burn has drop, store {
    minter: address,
    from: address,
    amount: u64
}

#[event]
struct Transfer has drop, store {
    from: address,
    to: address,
    amount: u64
}

#[event]
struct TransferFrom has drop, store {
    spender: address,
    from: address,
    to: address,
    amount: u64
}

#[event]
struct Balance has drop, store {
    account: address,
    amount: u64
}
```

```
    #[event]
    struct Approval has drop, store {
        owner: address,
        spender: address,
        amount: u64
    }

    #[event]
    struct ApprovalChange has drop, store {
        owner: address,
        spender: address,
        value: u64,
        increase: bool,
        amount: u64
    }

    #[event]
    struct WithdrawalCash has drop, store {
        sender: address,
        owner: address,
        amount: u64
    }

    const EUNAUTHORIZED: u64 = 10001;
    /// Balance Insufficient
    const EBALANCE_INSUFFICIENT:u64 =12001;
    /// Allowance Insufficient
    const EALLOWANCE_INSUFFICIENT:u64 = 12002;
    /// transfer check failed
    const ETRANSFER_CHECK_FAILED:u64 = 12003;
    /// balance useful Insufficient
    const EBALANCE_USEFUL_INSUFFICIENT:u64 = 12004;
    /// transfer account limit
    const ETRANSFER_ACCOUNT_LIMIT:u64 = 12005;
    /// Wallet disabled
    const EWALLET_DISABLE: u64 = 11005;
    /// When the authorization limit is reduced, the reduced value is greater than
the original value
    const EBANLANCE_AUTHORIZED_LIMIT: u64 = 11019;

    const ASSET_SYMBOL: vector<u8> = b"TD";

    #[view]
    public fun stablecoin_address(): address {
        object::create_object_address(&@td_stablecoin, ASSET_SYMBOL)
    }

    #[view]
    public fun metadata(): Object<Metadata> {
        object::address_to_object(stablecoin_address())
    }
```

```
    #[view]
    public fun is_bank(acc: address): bool {
        let is = wallet_manage::is_bank_address(acc);
        is
    }

    #[view]
    public fun name(): String {
        let asset = metadata();
        fungible_asset::name(asset)
    }

    #[view]
    public fun symbol(): String {
        let asset = metadata();
        fungible_asset::symbol(asset)
    }

    #[view]
    public fun decimals(): u8 {
        let asset = metadata();
        fungible_asset::decimals(asset)
    }

    #[view]
    public fun supply(): u128 {
        let asset = metadata();

        let supply = fungible_asset::supply(asset);
        let s = option::borrow(&supply);
        *s
    }

    #[view]
    public fun allowance(owner: address, spender: address): u64 acquires
StablecoinMetadata {
        let amount = allowance_internal(owner, spender);
        amount
    }

    #[view]
    public fun balance_of(acc: address): u64 {
        let asset = metadata();
        primary_fungible_store::balance(acc, asset)
    }

    #[view]
    public fun get_exchange(currency: String): u64 acquires StablecoinMetadata {
        let smd = borrow_global<StablecoinMetadata>(stablecoin_address());
        if (simple_map::contains_key(&smd.exchange, &currency)) {
            let exchange_rate = simple_map::borrow(&smd.exchange, &currency);
```

```
            *exchange_rate
        } else { 0 }
    }

    #[view]
    public fun get_metadata_object_address():address acquires StablecoinMetadata {
        let metadata = borrow_global<StablecoinMetadata>(stablecoin_address());
        metadata.metadata_object_address
    }

    #[view]
    public fun get_event_account_address():address acquires StablecoinMetadata {
        let metadata = borrow_global<StablecoinMetadata>(stablecoin_address());
        account::get_signer_capability_address(&metadata.event_signer_cap)
    }

    fun init_module(td_signer: &signer) {
        // Create the stablecoin with primary store support.
        let constructor_ref = &object::create_named_object(td_signer, ASSET_SYMBOL);
        primary_fungible_store::create_primary_store_enabled_fungible_asset(
            constructor_ref,
            option::none(),
            utf8(ASSET_SYMBOL), /* name */
            utf8(ASSET_SYMBOL), /* symbol */
            8, /* decimals */
            utf8(b"http://example.com/favicon.ico"), /* icon */
            utf8(b"http://example.com") /* project */
        );

        // Set ALL stores for the fungible asset to untransferable.
        fungible_asset::set_untransferable(constructor_ref);

        // All resources created will be kept in the asset metadata object.
        let metadata_object_signer = &object::generate_signer(constructor_ref);

        // Create mint/burn/transfer refs to allow creator to manage the stablecoin.
        move_to(
            metadata_object_signer,
            Management {
                extend_ref: object::generate_extend_ref(constructor_ref),
                mint_ref: fungible_asset::generate_mint_ref(constructor_ref),
                burn_ref: fungible_asset::generate_burn_ref(constructor_ref),
                transfer_ref:
fungible_asset::generate_transfer_ref(constructor_ref),
                mutate_metadata_ref:
fungible_asset::generate_mutate_metadata_ref(constructor_ref)
            }
        );
        let (event_account_signer,event_signer_cap) =
account::create_resource_account(td_signer,b"stablecoin_events");

        move_to(
```

```
            metadata_object_signer,
            StablecoinMetadata {
                has_repository: true,
                exchange: simple_map::create(),
                allowances: simple_map::create(),
                event_signer_cap,
                metadata_object_address:signer::address_of(metadata_object_signer),
            }
        );



        move_to(metadata_object_signer,StablecoinEventHandle{
            mint_events:account::new_event_handle<Mint>(&event_account_signer),
            burn_events:account::new_event_handle<Burn>(&event_account_signer),
            transfer_events:account::new_event_handle<Transfer>
(&event_account_signer),
            transfer_from_events:account::new_event_handle<TransferFrom>
(&event_account_signer),
            balance_events:account::new_event_handle<Balance>
(&event_account_signer),
            approval_events:account::new_event_handle<Approval>
(&event_account_signer),
            withdrawal_cash_events:account::new_event_handle<WithdrawalCash>
(&event_account_signer),
            approval_change_events:account::new_event_handle<ApprovalChange>
(&event_account_signer),
            set_stablecoin_info_events:account::new_event_handle<SetStablecoinInfo>
(&event_account_signer),
        })

    }



    public entry fun set_stablecoin_info(owner
:&signer,name:String,symbol:String,decimals:u8) acquires Management,
StablecoinEventHandle {
        assert_is_owner(owner);
        let mutate_metadata_ref = &borrow_global<Management>
(stablecoin_address()).mutate_metadata_ref;

        fungible_asset::mutate_metadata(
            mutate_metadata_ref,
            option::some(name),
            option::some(symbol),
            option::some(decimals),
            option::none(),
            option::none(),
        );

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
```

```
        event::emit_event(
            &mut events.set_stablecoin_info_events,
            SetStablecoinInfo { sender: signer::address_of(owner), name, symbol,
decimals});
    }

    public entry fun config_repository(bank: &signer, has: bool) acquires
StablecoinMetadata {
        assert_is_bank(bank);

        let smd = borrow_global_mut<StablecoinMetadata>(stablecoin_address());
        smd.has_repository = has;

    }

    // Only repository manage call.
    public(friend) fun mint_stablecoin(repository_address: address, amount: u64)
acquires Management,StablecoinEventHandle {
        mint_internal(repository_address, amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.mint_events,
            Mint { minter: repository_address, to: repository_address, amount });
    }

    public(friend) fun burn_stablecoin(repository_address: address, amount: u64)
acquires Management,StablecoinEventHandle {
        burn_internal(repository_address, amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.burn_events,
            Burn { minter: repository_address, from: repository_address, amount });

        // event::emit_event(
        //     &mut events.balance_events,
        //     Balance { account: repository_address, amount:
balance_of(repository_address) });

    }

    public(friend) fun transfer_stablecoin(repository_address: address, to: address,
amount: u64) acquires Management,StablecoinEventHandle {
        transfer_internal(repository_address, to, amount);
    }

    public entry fun set_exchange(sender: &signer,currency: String, rate: u64)
acquires StablecoinMetadata {

        assert_is_bank(sender);
        set_exchange_internal(currency, rate);
```

```
    }

    public entry fun mint(bank: &signer, to: address, amount: u64) acquires
Management,StablecoinEventHandle {
        assert_is_bank(bank);
        assert!(wallet_manage::get_wallet_status(to), EWALLET_DISABLE);

        mint_internal(to, amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.mint_events,
            Mint { minter: signer::address_of(bank), to, amount });
    }

    public entry fun transfer(from: &signer, to: address, amount: u64) acquires
Management,StablecoinEventHandle {
        let from_address = signer::address_of(from);

        transfer_internal(from_address, to, amount);
    }

    public entry fun transfer_from(spender: &signer, from: address, to: address,
amount: u64) acquires Management, StablecoinMetadata,StablecoinEventHandle {
        let spender_address = signer::address_of(spender);

        spend_allowance(from, spender_address, amount);

        transfer_internal(from, to, amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.transfer_from_events,
            TransferFrom { spender: spender_address, from, to, amount });

    }

    public entry fun approve(owner: &signer, spender: address, amount: u64) acquires
StablecoinMetadata,StablecoinEventHandle {

        let owner_address = signer::address_of(owner);
        approve_internal(owner_address, spender, amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.approval_events,
            Approval { owner: owner_address, spender, amount });
    }

    public entry fun approve_increase(owner: &signer, spender: address, amount: u64)
acquires StablecoinMetadata,StablecoinEventHandle {
```

```
        let owner_address = signer::address_of(owner);

        let old_amount = allowance_internal(owner_address, spender);
        let new_amount = old_amount + amount;

        approve_internal(owner_address, spender, new_amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.approval_events,
            Approval {
                owner: owner_address,
                spender,
                amount:new_amount });

        event::emit_event(
            &mut events.approval_change_events,
            ApprovalChange {
                owner: owner_address,
                spender,
                value:amount,
                increase:true,
                amount:new_amount });
    }

    public entry fun approve_decrease(owner: &signer, spender: address, amount: u64)
acquires StablecoinMetadata,StablecoinEventHandle {
        let owner_address = signer::address_of(owner);

        let old_amount = allowance_internal(owner_address, spender);
        assert!(old_amount >= amount, EBANLANCE_AUTHORIZED_LIMIT);

        let new_amount = old_amount - amount;
        approve_internal(owner_address, spender, new_amount);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.approval_events,
            Approval {
                owner: owner_address,
                spender,
                amount:new_amount
            });

        event::emit_event(
            &mut events.approval_change_events,
            ApprovalChange {
                owner: owner_address,
                spender,
                value:amount,
                increase:false,
                amount:new_amount
```

```
            });
    }

    public entry fun withdraw_by_user(account: &signer, amount: u64) acquires
Management, StablecoinMetadata,StablecoinEventHandle {
        let from = signer::address_of(account);

        if (has_repository()) {
            let repository_address = wallet_manage::repository_address();
            transfer_internal(from, repository_address, amount);
        } else {
            burn_internal(from, amount);
        };

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.withdrawal_cash_events,
            WithdrawalCash { sender: from, owner: from, amount });
    }

    public entry fun withdraw_by_bank(bank: &signer, to: address, amount: u64)
acquires Management, StablecoinMetadata,StablecoinEventHandle {
        assert_is_bank(bank);

        let bank_addr = signer::address_of(bank);
        if (has_repository()) {
            let repository_address = wallet_manage::repository_address();
            transfer_internal(to, repository_address, amount);
        } else {
            burn_internal(to, amount);
        };

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.withdrawal_cash_events,
            WithdrawalCash { sender: bank_addr, owner: to, amount });
    }

    /// Allow a spender to transfer tokens from the owner's account given their
signed approval.
    /// Caller needs to provide the from account's scheme and public key which can
be gotten via the Aptos SDK.
    public fun transfer_approva(
        spender: &signer,
        proof: vector<u8>,
        from: address,
        from_account_scheme: u8,
        from_public_key: vector<u8>,
        to: address,
        amount: u64
    ) acquires Management,StablecoinEventHandle {
```

```
        let expected_message = ApprovalMsg {
            owner: from,
            to,
            nonce: account::get_sequence_number(from),
            chain_id: chain_id::get(),
            spender: signer::address_of(spender),
            amount
        };
        account::verify_signed_message(from, from_account_scheme, from_public_key,
proof, expected_message);

        transfer_internal(from, to, amount);
    }

    fun set_exchange_internal(currency: String, rate: u64) acquires
StablecoinMetadata {
        let smd = borrow_global_mut<StablecoinMetadata>(stablecoin_address());

        if (simple_map::contains_key(&smd.exchange, &currency)) {
            let _exchange_rate = simple_map::borrow_mut(&mut smd.exchange,
&currency);
            _exchange_rate = &mut rate;
        } else {
            simple_map::add(&mut smd.exchange, currency, rate);
        };
    }

    fun spend_allowance(owner: address, spender: address, amount: u64) acquires
StablecoinMetadata {
        let currentAllowance = allowance_internal(owner, spender);
        assert!(currentAllowance >= amount, EALLOWANCE_INSUFFICIENT);
        approve_internal(owner, spender, currentAllowance - amount);
    }

    fun approve_internal(owner: address, spender: address, amount: u64) acquires
StablecoinMetadata {

        let smd = borrow_global_mut<StablecoinMetadata>(stablecoin_address());

        if (simple_map::contains_key(&smd.allowances, &owner)) {
            let approve = simple_map::borrow_mut(&mut smd.allowances, &owner);
            simple_map::upsert(approve, spender, amount);
        } else {
            let approve = simple_map::create<address, u64>();
            simple_map::add(&mut approve, spender, amount);
            simple_map::add(&mut smd.allowances, owner, approve);
        };

    }

    fun allowance_internal(owner: address, spender: address): u64 acquires
StablecoinMetadata {
```

```
        let smd = borrow_global_mut<StablecoinMetadata>(stablecoin_address());
        if (simple_map::contains_key(&smd.allowances, &owner)) {
            let approve = simple_map::borrow(&mut smd.allowances, &owner);
            if (simple_map::contains_key(approve, &spender)) {
                let m = simple_map::borrow(approve, &spender);
                return *m
            }
        };

        0
    }

    fun mint_internal(to: address, amount: u64) acquires
Management,StablecoinEventHandle {
        if (amount == 0) { return };

        let management = borrow_global<Management>(stablecoin_address());
        let tokens = fungible_asset::mint(&management.mint_ref, amount);
        // Ensure not to call pfs::deposit or dfa::deposit directly in the module.
        fungible_asset::deposit_with_ref(&management.transfer_ref,
primary_fungible_store::ensure_primary_store_exists(to, metadata()), tokens);

        assert_transfer_to_limit(to);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.transfer_events,
            Transfer { from: @0x00, to, amount });

        event::emit_event(
            &mut events.balance_events,
            Balance { account: to, amount: balance_of(to) });
    }

    fun burn_internal(from: address, amount: u64) acquires
Management,StablecoinEventHandle {

        let store = primary_fungible_store::ensure_primary_store_exists(from,
metadata());

        if (amount == 0) { return };

        let management = borrow_global<Management>(stablecoin_address());
        let tokens = fungible_asset::withdraw_with_ref(&management.transfer_ref,
store, amount);

        fungible_asset::burn(&management.burn_ref, tokens);

        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.transfer_events,
            Transfer { from , to:@0x00, amount });
```

```
        event::emit_event(
            &mut events.balance_events,
            Balance { account: from, amount: balance_of(from) });

    }

    fun transfer_internal(from: address, to: address, amount: u64) acquires
Management,StablecoinEventHandle {
        assert_transfer(from, to, amount);

        assert!(balance_of(from)>=amount,EBALANCE_INSUFFICIENT);
        let transfer_ref = &borrow_global<Management>
(stablecoin_address()).transfer_ref;
        primary_fungible_store::transfer_with_ref(transfer_ref, from, to, amount);

        assert_transfer_from_limit(from);
        assert_transfer_to_limit(to);


        let events = borrow_global_mut<StablecoinEventHandle>(stablecoin_address());
        event::emit_event(
            &mut events.transfer_events,
            Transfer { from, to, amount });

        event::emit_event(
            &mut events.balance_events,
            Balance { account: from, amount: balance_of(from) });

        event::emit_event(
            &mut events.balance_events,
            Balance { account: to, amount: balance_of(to) });

    }

    fun assert_transfer_from_limit(from: address) {
        if (from == wallet_manage::repository_address()) { return };

        let (_, _, _, freeze_balance) = wallet_manage::get_wallet_limit(from);
        let from_balance = balance_of(from);

        assert!(from_balance >= freeze_balance, EBALANCE_USEFUL_INSUFFICIENT)
    }

    fun assert_transfer_to_limit(to: address) {
        if (to == wallet_manage::repository_address()) { return };

        let (_, deposit_limit, _, _) = wallet_manage::get_wallet_limit(to);
        let to_balance = balance_of(to);

        assert!(to_balance <= deposit_limit, ETRANSFER_ACCOUNT_LIMIT);
    }
```

```
    fun assert_transfer(from: address, to: address, amount: u64) {
        let can = wallet_manage::check_allowed_transfer(from, to, amount);
        assert!(can, ETRANSFER_CHECK_FAILED);
    }

    fun assert_is_bank(bank: &signer) {
        let bank_addr = signer::address_of(bank);
        assert!(wallet_manage::is_bank_address(bank_addr), EUNAUTHORIZED);
    }

    fun assert_is_owner(owner: &signer) {
        let owner_addr = signer::address_of(owner);
        assert!(wallet_manage::owner()==owner_addr, EUNAUTHORIZED);
    }

    fun has_repository(): bool acquires StablecoinMetadata {
        let smd = borrow_global<StablecoinMetadata>(stablecoin_address());
        smd.has_repository
    }

    #[test_only]
    public fun init_for_test(td_signer: &signer) {
        init_module(td_signer);
    }
}
```

Transfer

Method

```
public entry fun transfer(from: &signer, to: address, amount: u64)
```

Parameter Description

```
to: the wallet address that accepts the transfer
amount: the amount of Tokens to be transferred out
```

Event

```
#[event]
struct Transfer has drop, store {
    from: address,
    to: address,
    amount: u64
}
#[event]
struct Balance has drop, store {
    account: address,
    amount: u64
}
```

Authorized Transfer

Method

```
public entry fun transfer_from(spender: &signer, from: address, to: address, amount:
u64)
```

Parameter Description

```
from: the wallet address that transfers out Tokens
to: the wallet address that accepts the transfer
amount: the amount of Tokens to be transferred
```

Event

```
#[event]
struct Transfer has drop, store {
    from: address,
    to: address,
    amount: u64
}
#[event]
struct Balance has drop, store {
    account: address,
    amount: u64
}
#[event]
struct TransferFrom has drop, store {
    spender: address,
    from: address,
    to: address,
    amount: u64
}
```

Authorization

Method

```
public entry fun approve(owner: &signer, spender: address, amount: u64)
```

Parameter Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the authorized amount
```

Event

```
#[event]
struct Approval has drop, store {
    owner: address,
    spender: address,
    amount: u64
}
```

Increase Authorization

Method

```
public entry fun approve_increase(owner: &signer, spender: address, amount: u64)
```

Parameter Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the amount of increased authorization
```

Event

```
#[event]
struct Approval has drop, store {
    owner: address,
    spender: address,
    amount: u64
}
#[event]
struct ApprovalChange has drop, store {
    owner: address,
    spender: address,
    value: u64,
    increase: bool,
    amount: u64
}
```

Decrease Authorization

Method

```
public entry fun approve_decrease(owner: &signer, spender: address, amount: u64)
```

Parameter Description

```
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Token
amount: the amount of decreased authorization
```

Event

```
#[event]
struct Approval has drop, store {
    owner: address,
    spender: address,
    amount: u64
}
#[event]
struct ApprovalChange has drop, store {
    owner: address,
    spender: address,
    value: u64,
    increase: bool,
    amount: u64
}
```

Authorization Query

Method

```
#[view]
public fun allowance(owner: address, spender: address): u64
```

Parameter Description

```
owner: the authorizer's wallet address
spender: the delegated wallet address with transfer authorization and able to
tansfer authorizer's authorized Tokens
Returns the remaining amount of authorized transferrable Tokens
```

Balance Query

Method

```
#[view]
public fun balance_of(acc: address): u64
```

Parameter Description

```
acc: the wallet address to be queried
Returns the balance of the wallet
```

User Withdrawal

Method

```
public entry fun withdraw_by_user(account: &signer, amount: u64)
```

Parameter Description

```
amount: the amount of the user's withdrawal
Return whether the result is successful
```

Event

```
 #[event]
struct WithdrawalCash has drop, store {
    sender: address,
    owner: address,
    amount: u64
}
#[event]
struct Transfer has drop, store {
    from: address,
    to: address,
    amount: u64
}
#[event]
struct Balance has drop, store {
    account: address,
    amount: u64
}
```

2.2.2.2 Token Wallet Contract Interfaces

Interface Files

```
module td_stablecoin::wallet_manage {

    friend td_stablecoin::stablecoin;

    use std::bcs;
    use std::signer;
    use std::simple_map::{Self, SimpleMap};
    use std::string;
    use std::string::String;
    use std::vector;
    use aptos_std::smart_vector::{Self, SmartVector};
    use aptos_framework::event;
    use aptos_framework::object;
    use aptos_framework::timestamp;
    use aptos_framework::account;

    const WALLET_MANAGE_SYMBOL: vector<u8> = b"wallet";
    const REPOSITORY_SYMBOL: vector<u8> = b"repository";
    const SERVICE_PROVIDER_SYMBOL: vector<u8> = b"ServiceProvider";

    const MANAGE_RESOURCE_SYMBOL: vector<u8> = b"manage_resource";
    const WALLET_RESOURCE_SYMBOL: vector<u8> = b"wallet_resource";

    /// Caller is not authorized to make this call
    const EUNAUTHORIZED: u64 = 10001;
    /// Invalid address
    const EADDRESS_INVALID: u64 = 10002;
    /// Invalid operation
    const EOPERATE_INVALID: u64 = 10003;
    /// The stablecoin amount must be greater than 0
    const EBANLANCE_ZERO: u64 = 10004;
    /// The account address already exists
    const EACCOUNT_EXIST: u64 = 11001;
    /// The account address does not exist
    const EACCOUNT_NOT_EXIST: u64 = 11002;
    /// Wallet type error
    const EWALLET_TYPE: u64 = 11003;
    /// Wallet type status error
    const EWALLET_TYPE_STATUS: u64 = 11004;
    /// Wallet disabled
    const EWALLET_DISABLE: u64 = 11005;
    /// The wallet does not exist
    const EWALLET_NOT_EXIST: u64 = 11006;
    /// From address and to address cannot be consistent
    const EFROM_TO_EQUAL: u64 = 11007;
    /// Single transfer exceeding the limit
    const ETRANS_LIMIT: u64 = 11008;
    /// The daily transfer limit has exceeded
```

```
    const ETRANS_DAILY_LIMIT: u64 = 11009;
    /// Invalid wallet type
    const EWALLET_TYPE_INVALID: u64 = 11010;
    /// The SP already exists
    const ESP_EXIST: u64 = 11011;
    /// The SP does not exist
    const ESP_NOT_EXIST: u64 = 11012;
    /// The number of array type parameters does not match or is 0
    const EARGS_LENGHT: u64 = 11013;
    /// This account has already been registered and used by another SP
    const EWALLET_EXIST: u64 = 11014;
    /// The sender does not have permission to register wallet methods
    const ESENDER_UNAUTHORIZED: u64 = 11015;
    /// When using the contract to call the registered wallet method, the contract
address and the signer address are not from the same SP organization
    const ESP_ADDRESS: u64 = 11016;
    /// Wallet type not authorized for this SP organization to register wallet
    const ESP_WALLET_TYPE_UNAUTHORIZED: u64 = 11017;
    /// SP organization has been disabled
    const ESP_DISABLE: u64 = 11018;
    /// When the authorization limit is reduced, the reduced value is greater than
the original value
    const EBANLANCE_AUTHORIZED_LIMIT: u64 = 11019;
    /// SP registration is prohibited
    const ESP_REGISTER_DISABLE: u64 = 11020;
    /// Account balance reaches the upper limit
    const ETRANS_DEPOSIT_LIMIT: u64 = 11022;
    /// The wallet type already exists
    const EWALLET_TYPE_EXIST: u64 = 11023;

    /// Contract disabled
    const EPAUSED: u64 = 11404;

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct Roles has key {
        owner: address,
        bank_accounts: SmartVector<address>,
        operation_accounts: SmartVector<address>,
        sp_accounts: SimpleMap<address, u64>,
        resource_signer_cap: account::SignerCapability,
        metadata_object_address:address,
    }

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct WalletEventHandle has key {
        set_operation_addr_events : event::EventHandle<SetOperationAddr>,
        del_operation_addr_events : event::EventHandle<DelOperationAddr>,
        set_bank_addr_events : event::EventHandle<SetBankAddr>,
        del_bank_addr_events : event::EventHandle<DelBankAddr>,
        set_register_status_events : event::EventHandle<SetRegisterStatus>,
        set_td_status_events : event::EventHandle<SetTDStatus>,
        set_wallet_type_info_events : event::EventHandle<SetWalletTypeInfo>,
```

```
        set_wallet_type_status_events : event::EventHandle<SetWalletTypeStatus>,
        register_service_provider_organisation_events :
event::EventHandle<RegisterServiceProviderOrganisation>,
        update_service_provider_code_events :
event::EventHandle<UpdateServiceProviderCode>,
        update_service_provider_organisation_wallet_types_events :
event::EventHandle<UpdateServiceProviderOrganisationWalletTypes>,
        update_service_provider_organisation_status_events :
event::EventHandle<UpdateServiceProviderOrganisationStatus>,
        set_service_provider_account_events :
event::EventHandle<SetServiceProviderAccount>,
        set_wallet_status_events : event::EventHandle<SetWalletStatus>,
        set_wallet_control_amount_events :
event::EventHandle<SetWalletControlAmount>,
        cancel_wallet_events : event::EventHandle<CancelWallet>,
        set_wallet_limit_events : event::EventHandle<SetWalletLimit>,
        cencel_wallet_limit_events : event::EventHandle<CencelWalletLimit>,
        register_wallet_events : event::EventHandle<RegisterWallet>,
        update_wallet_type_events : event::EventHandle<UpdateWalletType>,
    }

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct ServiceProviders has key {
        last_id: u64,
        service_provider_codes: SimpleMap<String, u64>,
        service_providers: SimpleMap<u64, ServiceProvider>
    }

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct ServiceProvider has key, store {
        id: u64,
        code: String,
        status: bool,
        wallet_types: SmartVector<u64>
    }

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct WalletTypes has key, store {
        last_id: u64,
        wallet_type_list: SimpleMap<u64, WalletTypeInfo>
    }

    const STATUS_ENABLED: u64 = 1;
    const STATUS_DISABLED: u64 = 2;

    #[resource_group_member(group = aptos_framework::object::ObjectGroup)]
    struct WalletTypeInfo has key, store {
        name: String,
        status: u64,
        deposit_limit: u64,
        daily_trans_limit: u64,
        per_trans_limit: u64
```

```
}

#[resource_group_member(group = aptos_framework::object::ObjectGroup)]
struct WalletInfo has key, store, drop {
    wallet_type_id: u64,
    status: u64,
    // true: use wallet type limit,default
    // false: user wallet limit
    is_use_default: bool,
    deposit_limit: u64,
    daily_trans_limit: u64,
    per_trans_limit: u64,
    freeze_balance: u64,
    creator: address,
    sp_id: u64
}

#[resource_group_member(group = aptos_framework::object::ObjectGroup)]
struct WalletTrans has key,store,drop {
    day_id: u64,
    amount: u64
}

#[resource_group_member(group = aptos_framework::object::ObjectGroup)]
struct State has key {
    contract_status: bool,
    register_status: bool
}

struct WalletLimit has store, drop {
    deposit_limit: u64,
    daily_trans_limit: u64,
    per_trans_limit: u64,
    freeze_balance: u64
}

#[event]
struct SetOperationAddr has drop, store {
    sender: address,
    operation_address: address
}

#[event]
struct DelOperationAddr has drop, store {
    sender: address,
    operation_address: address
}

#[event]
struct SetBankAddr has drop, store {
    sender: address,
    bank_address: address
```

```
}

#[event]
struct DelBankAddr has drop, store {
    sender: address,
    bank_address: address
}

#[event]
struct SetRegisterStatus has drop, store {
    sender: address,
    status: bool
}

#[event]
struct SetTDStatus has drop, store {
    sender: address,
    status: bool
}

#[event]
struct SetWalletTypeInfo has drop, store {
    sender: address,
    wallet_type_id: u64,
    name: String,
    status: u64,
    deposit_limit: u64,
    daily_trans_limit: u64,
    per_trans_limit: u64
}

#[event]
struct SetWalletStatus has drop, store {
    sender: address,
    wallet_address: address,
    status: u64
}

#[event]
struct SetWalletControlAmount has drop, store {
    sender: address,
    wallet_address: address,
    freeze_balance: u64
}

#[event]
struct CancelWallet has drop, store {
    sender: address,
    wallet_address: address
}
```

```
    #[event]
    struct SetWalletTypeStatus has drop, store {
        sender: address,
        wallet_type_id: u64,
        status: u64
    }

    #[event]
    struct RegisterWallet has drop, store {
        sender: address,
        wallet_address: address,
        wallet_type_id: u64,
        sp_code: String
    }

    #[event]
    struct UpdateWalletType has drop, store {
        sender: address,
        wallet_address: address,
        wallet_type_id: u64
    }

    #[event]
    struct RegisterServiceProviderOrganisation has drop, store {
        sender: address,
        code: String,
        sp_id: u64,
        wallet_types: vector<u64>,
        account: vector<address>
    }

    #[event]
    struct UpdateServiceProviderCode has drop, store {
        sender: address,
        oldCode: String,
        newCode: String
    }

    #[event]
    struct UpdateServiceProviderOrganisationWalletTypes has drop, store {
        sender: address,
        code: String,
        sp_id: u64,
        sp_wallet_types: vector<u64>,
        sp_wallet_type_auths: vector<bool>
    }

    #[event]
    struct UpdateServiceProviderOrganisationStatus has drop, store {
        sender: address,
        code: String,
        sp_id: u64,
```

```
    status: bool
}

#[event]
struct SetServiceProviderAccount has drop, store {
    sender: address,
    code: String,
    sp_id: u64,
    sp_accounts: vector<address>,
    sp_accounts_auth: vector<bool>
}

#[event]
struct SetWalletLimit has drop,store {
    operator: address,
    wallet_address: address,
    deposit_limit: u64,
    daily_trans_limit: u64,
    per_trans_limit: u64
}

#[event]
struct CencelWalletLimit has drop,store {
    operator: address,
    wallet_address: address
}

#[event]
struct TransferOwnership has drop, store {
    previous_owner: address,
    new_owner: address
}

#[view]
public fun wallet_manage_address(): address {
    object::create_object_address(&@td_stablecoin, WALLET_MANAGE_SYMBOL)
}

#[view]
public fun owner(): address acquires Roles {
    let roles = borrow_global<Roles>(wallet_manage_address());
    roles.owner
}

#[view]
public fun repository_address(): address {
    object::create_object_address(&@td_stablecoin, REPOSITORY_SYMBOL)
}

#[view]
public fun get_operation_address(): vector<address> acquires Roles {
    let roles = borrow_global<Roles>(wallet_manage_address());
```

```
        let ops = smart_vector::to_vector(&roles.operation_accounts);
        ops
    }

    #[view]
    public fun is_operation_address(account: address): bool acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let has = smart_vector::contains(&roles.operation_accounts, &account);
        has
    }

    #[view]
    public fun get_bank_address(): vector<address> acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let ops = smart_vector::to_vector(&roles.bank_accounts);
        ops
    }

    #[view]
    public fun is_bank_address(account: address): bool acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let has = smart_vector::contains(&roles.bank_accounts, &account);
        has
    }

    #[view]
    public fun get_contract_status(): bool acquires State {
        let status = borrow_global<State>(wallet_manage_address());
        status.contract_status
    }

    #[view]
    public fun get_register_status(): bool acquires State {
        let status = borrow_global<State>(wallet_manage_address());
        status.register_status
    }

    #[view]
    public fun get_wallet_type_info(wallet_type_id: u64): (String, u64, u64, u64,
u64) acquires WalletTypes {
        let wallet_types = borrow_global<WalletTypes>(wallet_manage_address());
        assert!(simple_map::contains_key(&wallet_types.wallet_type_list,
&wallet_type_id), EWALLET_TYPE_INVALID);

        let wallet_type = simple_map::borrow(&wallet_types.wallet_type_list,
&wallet_type_id);
        (wallet_type.name, wallet_type.status, wallet_type.deposit_limit,
wallet_type.daily_trans_limit, wallet_type.per_trans_limit)
    }

    #[view]
```

```
    public fun get_wallet_status(wallet_address: address): bool acquires
Roles,WalletInfo {

        if (!exists_wallet(wallet_address)) { false }
        else {
            let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
            wallet_info.status == STATUS_ENABLED
        }
    }

    #[view]
    public fun get_service_provider(sp_code: String): (u64, String, vector<u64>,
bool) acquires ServiceProviders {

        let server_providers = borrow_global<ServiceProviders>
(wallet_manage_address());
        assert!
(simple_map::contains_key(&server_providers.service_provider_codes,&sp_code),ESP_NOT
_EXIST);

        let sp_code1 = sp_code;
        let sp_id = simple_map::borrow(&server_providers.service_provider_codes,
&sp_code1);
        let sp_info = simple_map::borrow(&server_providers.service_providers,
sp_id);
        (sp_info.id, sp_info.code, smart_vector::to_vector(&sp_info.wallet_types),
sp_info.status)

    }

    #[view]
    public fun get_service_provider_wallet_types(sp_code: String): vector<u64>
acquires ServiceProviders {

        let server_providers = borrow_global<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_NOT_EXIST);

        let sp_code1 = sp_code;
        let sp_id = simple_map::borrow(&server_providers.service_provider_codes,
&sp_code1);
        let sp_info = simple_map::borrow(&server_providers.service_providers,
sp_id);
        let wts = smart_vector::to_vector(&sp_info.wallet_types);
        wts
    }

    #[view]
    public fun get_account_sp_code(sp_account: address): String acquires Roles,
ServiceProviders {
```

```
        let roles = borrow_global<Roles>(wallet_manage_address());
        assert!(simple_map::contains_key(&roles.sp_accounts, &sp_account),
EWALLET_NOT_EXIST);

        let sp_id = simple_map::borrow(&roles.sp_accounts, &sp_account);
        let sps = borrow_global<ServiceProviders>(wallet_manage_address());
        assert!(simple_map::contains_key(&sps.service_providers, sp_id),
ESP_NOT_EXIST);

        let sp = simple_map::borrow(&sps.service_providers, sp_id);
        assert!(sp.status, ESP_DISABLE);
        sp.code

    }

    #[view]
    public fun get_metadata_object_address():address acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        roles.metadata_object_address
    }

    #[view]
    public fun get_wallet_limit(wallet_address: address): (u64, u64, u64, u64)
acquires Roles,WalletInfo, WalletTypes {
        check_wallet(wallet_address);
        let limit = get_wallet_limit_internal(wallet_address);
        (limit.per_trans_limit, limit.deposit_limit, limit.daily_trans_limit,
limit.freeze_balance)
    }

    #[view]
    public fun get_wallet(wallet_address: address): (u64, u64, bool, u64, u64, u64,
u64, address, u64) acquires Roles,WalletInfo {
        check_wallet(wallet_address);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        (
            wallet_info.wallet_type_id,
            wallet_info.status,
            wallet_info.is_use_default,
            wallet_info.deposit_limit,
            wallet_info.daily_trans_limit,
            wallet_info.per_trans_limit,
            wallet_info.freeze_balance,
            wallet_info.creator,
            wallet_info.sp_id
        )

    }
```

```
    #[view]
    public fun get_wallet_of_sp(wallet_address: address):u64 acquires
Roles,WalletInfo {
        check_wallet(wallet_address);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.sp_id
    }

    #[view]
    public fun get_wallet_type(wallet_address: address): u64 acquires
Roles,WalletInfo {
        check_wallet(wallet_address);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.wallet_type_id

    }
    #[view]
    public fun get_event_account_address():address acquires Roles {
        let metadata = borrow_global<Roles>(wallet_manage_address());
        account::get_signer_capability_address(&metadata.resource_signer_cap)
    }

    fun init_module(wallet_signer: &signer) {
        let constructor_ref = &object::create_named_object(wallet_signer,
WALLET_MANAGE_SYMBOL);
        let metadata_object_signer = &object::generate_signer(constructor_ref);

        let (resource_account_signer, resource_signer_cap) =
account::create_resource_account(wallet_signer,MANAGE_RESOURCE_SYMBOL);
        move_to(
            metadata_object_signer,
            Roles {
                owner: @owner,
                bank_accounts: smart_vector::new(),
                operation_accounts: smart_vector::new(),
                sp_accounts: simple_map::create(),
                resource_signer_cap,
                metadata_object_address:signer::address_of(metadata_object_signer),
            }
        );

        move_to(
            metadata_object_signer,
            ServiceProviders {
                last_id: 1,
                service_provider_codes: simple_map::create(),
                service_providers: simple_map::create()
            }
```

```
        );

        move_to(metadata_object_signer, WalletTypes { last_id: 0, wallet_type_list:
simple_map::create() });

        move_to(metadata_object_signer, State { contract_status: true,
register_status: true });

        move_to(metadata_object_signer,WalletEventHandle{
            set_operation_addr_events:account::new_event_handle<SetOperationAddr>
(&resource_account_signer),
            del_operation_addr_events:account::new_event_handle<DelOperationAddr>
(&resource_account_signer),
            set_bank_addr_events:account::new_event_handle<SetBankAddr>
(&resource_account_signer),
            del_bank_addr_events:account::new_event_handle<DelBankAddr>
(&resource_account_signer),
            set_register_status_events:account::new_event_handle<SetRegisterStatus>
(&resource_account_signer),
            set_td_status_events:account::new_event_handle<SetTDStatus>
(&resource_account_signer),
            set_wallet_type_info_events:account::new_event_handle<SetWalletTypeInfo>
(&resource_account_signer),

set_wallet_type_status_events:account::new_event_handle<SetWalletTypeStatus>
(&resource_account_signer),

register_service_provider_organisation_events:account::new_event_handle<RegisterServ
iceProviderOrganisation>(&resource_account_signer),

update_service_provider_code_events:account::new_event_handle<UpdateServiceProviderC
ode>(&resource_account_signer),

update_service_provider_organisation_wallet_types_events:account::new_event_handle<U
pdateServiceProviderOrganisationWalletTypes>(&resource_account_signer),

update_service_provider_organisation_status_events:account::new_event_handle<UpdateS
erviceProviderOrganisationStatus>(&resource_account_signer),

set_service_provider_account_events:account::new_event_handle<SetServiceProviderAcco
unt>(&resource_account_signer),
            set_wallet_status_events:account::new_event_handle<SetWalletStatus>
(&resource_account_signer),

set_wallet_control_amount_events:account::new_event_handle<SetWalletControlAmount>
(&resource_account_signer),
            cancel_wallet_events:account::new_event_handle<CancelWallet>
(&resource_account_signer),
            set_wallet_limit_events:account::new_event_handle<SetWalletLimit>
(&resource_account_signer),
            cencel_wallet_limit_events:account::new_event_handle<CencelWalletLimit>
(&resource_account_signer),
```

```
            register_wallet_events:account::new_event_handle<RegisterWallet>
(&resource_account_signer),
            update_wallet_type_events:account::new_event_handle<UpdateWalletType>
(&resource_account_signer),
        })
    }

    public entry fun transfer_ownership(owner: &signer, new_owner: address) acquires
Roles {
        assert_is_owner(owner);
        let roles = borrow_global_mut<Roles>(wallet_manage_address());
        roles.owner = new_owner
    }

    public entry fun set_operation_address(owner: &signer, account: address)
acquires Roles,WalletEventHandle {
        assert_is_owner(owner);

        let roles = borrow_global_mut<Roles>(wallet_manage_address());
        if (smart_vector::contains(&roles.operation_accounts, &account)) { return };

        smart_vector::push_back(&mut roles.operation_accounts, account);

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_operation_addr_events,
            SetOperationAddr { sender: signer::address_of(owner), operation_address:
account });
    }

    public entry fun del_operation_address(owner: &signer, account: address)
acquires Roles,WalletEventHandle {
        assert_is_owner(owner);

        let owner_address = signer::address_of(owner);
        let roles = borrow_global_mut<Roles>(wallet_manage_address());

        let (has, index) = smart_vector::index_of(&roles.operation_accounts,
&account);
        assert!(has, EACCOUNT_NOT_EXIST);

        smart_vector::remove(&mut roles.operation_accounts, index);

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.del_operation_addr_events,
            DelOperationAddr { sender: owner_address, operation_address: account });
    }

    public entry fun set_bank_address(owner: &signer, account: address) acquires
Roles,WalletEventHandle {
        assert_is_owner(owner);
```

```
        let owner_address = signer::address_of(owner);
        let roles = borrow_global_mut<Roles>(wallet_manage_address());
        if (smart_vector::contains(&roles.bank_accounts, &account)) { return };

        smart_vector::push_back(&mut roles.bank_accounts, account);

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_bank_addr_events,
            SetBankAddr { sender: owner_address, bank_address: account });
    }

    public entry fun del_bank_address(owner: &signer, account: address) acquires
Roles,WalletEventHandle {
        assert_is_owner(owner);

        let owner_address = signer::address_of(owner);
        let roles = borrow_global_mut<Roles>(wallet_manage_address());

        let (has, index) = smart_vector::index_of(&roles.bank_accounts, &account);
        assert!(has, EACCOUNT_NOT_EXIST);

        smart_vector::remove(&mut roles.bank_accounts, index);

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.del_bank_addr_events,
            DelBankAddr { sender: owner_address, bank_address: account });
    }

    public entry fun set_register_status(bank: &signer, status: bool) acquires
Roles, State, WalletEventHandle {
        assert_is_bank(bank);

        let state = borrow_global_mut<State>(wallet_manage_address());
        state.register_status = status;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_register_status_events,
            SetRegisterStatus { sender: signer::address_of(bank), status });
    }

    public entry fun set_td_status(bank: &signer, status: bool) acquires Roles,
State, WalletEventHandle {
        assert_is_bank(bank);

        let state = borrow_global_mut<State>(wallet_manage_address());
        state.contract_status = status;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
```

```
        event::emit_event(
            &mut events.set_td_status_events,
            SetTDStatus { sender: signer::address_of(bank), status });
    }

    public entry fun set_wallet_type(
        bank: &signer,
        wallet_type_id: u64,
        name: String,
        deposit_limit: u64,
        daily_trans_limit: u64,
        per_trans_limit: u64
    ) acquires Roles, WalletTypes, WalletEventHandle {
        assert_is_bank(bank);

        let wallet_types = borrow_global_mut<WalletTypes>(wallet_manage_address());
        if (simple_map::contains_key(&wallet_types.wallet_type_list,
&wallet_type_id)) {
            let wallet_type = simple_map::borrow_mut(&mut
wallet_types.wallet_type_list, &wallet_type_id);
            wallet_type.name = name;
            wallet_type.deposit_limit = deposit_limit;
            wallet_type.daily_trans_limit = daily_trans_limit;
            wallet_type.per_trans_limit = per_trans_limit;
        } else {
            simple_map::add(
                &mut wallet_types.wallet_type_list,
                wallet_type_id,
                WalletTypeInfo { name, deposit_limit, daily_trans_limit,
per_trans_limit, status: STATUS_ENABLED }
            );
        };

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_wallet_type_info_events,
            SetWalletTypeInfo {
                sender: signer::address_of(bank),
                wallet_type_id,
                name,
                deposit_limit,
                daily_trans_limit,
                per_trans_limit,
                status: STATUS_ENABLED
            });

    }

    public entry fun set_wallet_type_status(bank: &signer, wallet_type_id: u64,
status: u64) acquires Roles, WalletTypes, WalletEventHandle {
        assert_is_bank(bank);
```

```
        assert!(status == STATUS_ENABLED || status == STATUS_DISABLED,
EWALLET_TYPE_STATUS);

        let wallet_types = borrow_global_mut<WalletTypes>(wallet_manage_address());
        assert!(simple_map::contains_key(&wallet_types.wallet_type_list,
&wallet_type_id), EWALLET_TYPE);

        let wallet_type = simple_map::borrow_mut(&mut wallet_types.wallet_type_list,
&wallet_type_id);
        wallet_type.status = status;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_wallet_type_status_events,
            SetWalletTypeStatus { sender: signer::address_of(bank), wallet_type_id,
status });
    }

    public entry fun register_service_provider(
        bank: &signer, sp_code: String, sp_wallet_types: vector<u64>, accounts:
vector<address>
    ) acquires Roles, WalletTypes, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global_mut<ServiceProviders>
(wallet_manage_address());
        assert!(!simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_EXIST);

        let last_id = server_providers.last_id;
        server_providers.last_id = server_providers.last_id + 1;

        let owner_wallet_types = borrow_global<WalletTypes>
(wallet_manage_address());
        vector::for_each(
            sp_wallet_types,
            |t| {
                assert!
(simple_map::contains_key(&owner_wallet_types.wallet_type_list, &t),
EWALLET_TYPE_INVALID);
            }
        );

        let roles = borrow_global_mut<Roles>(wallet_manage_address());
        vector::for_each(
            accounts,
            |a| {
                assert!(!simple_map::contains_key(&roles.sp_accounts, &a),
EWALLET_EXIST);

                // add sp account
                simple_map::add(&mut roles.sp_accounts, a, last_id);
```

```
            }
        );

        // add sp
        let sp_wallet_types1 = smart_vector::new<u64>();
        smart_vector::add_all(&mut sp_wallet_types1, sp_wallet_types);

        simple_map::add(&mut server_providers.service_provider_codes, sp_code,
last_id);
        simple_map::add(
            &mut server_providers.service_providers,
            last_id,
            ServiceProvider { id: last_id, code: sp_code, status: true,
wallet_types: sp_wallet_types1 }
        );

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.register_service_provider_organisation_events,
            RegisterServiceProviderOrganisation {
                sender: signer::address_of(bank),
                code: sp_code,
                sp_id: last_id,
                wallet_types: sp_wallet_types,
                account: accounts
            });

    }

    public entry fun update_service_provider_code(bank: &signer, sp_old_code:
String, sp_new_code: String) acquires Roles, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global_mut<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_old_code), ESP_NOT_EXIST);
        assert!(!simple_map::contains_key(&server_providers.service_provider_codes,
&sp_new_code), ESP_EXIST);

        let (_, id) = simple_map::remove(&mut
server_providers.service_provider_codes, &sp_old_code);

        simple_map::add(&mut server_providers.service_provider_codes, sp_new_code,
id);

        let sp_info = simple_map::borrow_mut(&mut
server_providers.service_providers, &id);
        sp_info.code = sp_new_code;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
```

```
            &mut events.update_service_provider_code_events,
            UpdateServiceProviderCode { sender: signer::address_of(bank), oldCode:
sp_old_code, newCode: sp_new_code });

    }

    public entry fun update_service_provider_wallet_type(
        bank: &signer, sp_code: String, sp_wallet_types: vector<u64>,
sp_wallet_type_auths: vector<bool>
    ) acquires Roles, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global_mut<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_NOT_EXIST);

        assert!(vector::length(&sp_wallet_types) ==
vector::length(&sp_wallet_type_auths), EARGS_LENGHT);

        let sp_code1 = sp_code;
        let sp_id = simple_map::borrow(&server_providers.service_provider_codes,
&sp_code1);
        let sp_info = simple_map::borrow_mut(&mut
server_providers.service_providers, sp_id);

        vector::enumerate_ref(
            &sp_wallet_types,
            |i, wt| {
                let auth = vector::borrow(&sp_wallet_type_auths, i);
                if (*auth) {
                    // add
                    if (!smart_vector::contains(&sp_info.wallet_types, wt)) {
                        smart_vector::push_back(&mut sp_info.wallet_types, *wt);
                    }
                } else {
                    // delete
                    let (has, i) = smart_vector::index_of(&sp_info.wallet_types,
wt);
                    if (has) {
                        smart_vector::remove(&mut sp_info.wallet_types, i);
                    }
                }
            }
        );

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.update_service_provider_organisation_wallet_types_events,
            UpdateServiceProviderOrganisationWalletTypes { sender:
signer::address_of(bank), code: sp_code, sp_id: *sp_id, sp_wallet_types,
sp_wallet_type_auths });
```

```
    }

    public entry fun update_service_provider_status(bank: &signer, sp_code: String,
status: bool) acquires Roles, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global_mut<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_NOT_EXIST);

        let sp_code1 = sp_code;
        let sp_id = simple_map::borrow(&server_providers.service_provider_codes,
&sp_code1);
        let sp_info = simple_map::borrow_mut(&mut
server_providers.service_providers, sp_id);
        sp_info.status = status;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.update_service_provider_organisation_status_events,
            UpdateServiceProviderOrganisationStatus { sender:
signer::address_of(bank), code: sp_code, sp_id: *sp_id, status });
    }

    public entry fun set_service_provider_account(
        bank: &signer, sp_code: String, sp_accounts: vector<address>,
sp_accounts_auth: vector<bool>
    ) acquires Roles, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_NOT_EXIST);

        assert!(vector::length(&sp_accounts) == vector::length(&sp_accounts_auth),
EARGS_LENGHT);

        let sp_code1 = sp_code;
        let sp_id = simple_map::borrow(&server_providers.service_provider_codes,
&sp_code1);

        let roles = borrow_global_mut<Roles>(wallet_manage_address());
        vector::enumerate_ref(
            &sp_accounts,
            |i, acc| {
                let auth = vector::borrow(&sp_accounts_auth, i);
                if (*auth) {
                    // add
                    if (!simple_map::contains_key(&roles.sp_accounts, acc)) {
```

```
                        simple_map::add(&mut roles.sp_accounts, *acc, *sp_id);
                    } else {
                        assert!(simple_map::borrow(&roles.sp_accounts, acc) ==
sp_id, EWALLET_EXIST)
                    }
                } else {
                    // delete

                    if (simple_map::contains_key(&roles.sp_accounts, acc)) {
                        simple_map::remove(&mut roles.sp_accounts, acc);

                    }
                }
            }
        );

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_service_provider_account_events,
            SetServiceProviderAccount { sender: signer::address_of(bank), code:
sp_code, sp_id: *sp_id, sp_accounts, sp_accounts_auth });
    }

    public entry fun set_wallet_status(bank: &signer, wallet_address: address,
status: u64, _data: String) acquires Roles,WalletInfo, WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);
        check_status_type(status);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.status = status;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_wallet_status_events,
            SetWalletStatus { sender: signer::address_of(bank), wallet_address,
status });
    }

    public entry fun set_wallet_control_amount(bank: &signer, wallet_address:
address, amount: u64, _data: String) acquires Roles, WalletInfo,WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.freeze_balance = amount;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_wallet_control_amount_events,
```

```
            SetWalletControlAmount { sender: signer::address_of(bank),
wallet_address, freeze_balance: amount });
    }

    public entry fun cancel_wallet(bank: &signer, wallet_address: address, _data:
String) acquires Roles,WalletInfo,WalletTrans, WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);

        let wallet_resource_account_address
=get_wallet_resource_account_address(wallet_address);
        move_from<WalletInfo>(wallet_resource_account_address);
        move_from<WalletTrans>(wallet_resource_account_address);

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.cancel_wallet_events,
            CancelWallet { sender: signer::address_of(bank), wallet_address });
    }

    public entry fun set_wallet_limit(
        bank: &signer, wallet_address: address, deposit_limit: u64,
daily_trans_limit: u64, per_trans_limit: u64
    ) acquires Roles,WalletInfo, WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);

        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.is_use_default = false;
        wallet_info.deposit_limit = deposit_limit;
        wallet_info.daily_trans_limit = daily_trans_limit;
        wallet_info.per_trans_limit = per_trans_limit;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.set_wallet_limit_events,
            SetWalletLimit{
                operator:signer::address_of(bank),
                wallet_address,
                deposit_limit,
                daily_trans_limit,
                per_trans_limit
            } )
    }



    public entry fun cancel_wallet_limit(bank: &signer, wallet_address: address)
acquires Roles,WalletInfo, WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);
```

```
        let wallet_info = borrow_global_mut<WalletInfo>
(get_wallet_resource_account_address(wallet_address));
        wallet_info.is_use_default = true;
        wallet_info.deposit_limit = 0;
        wallet_info.daily_trans_limit = 0;
        wallet_info.per_trans_limit = 0;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.cencel_wallet_limit_events,
            CencelWalletLimit{
                operator:signer::address_of(bank),
                wallet_address,
            });
    }

    public entry fun update_service_provider_info(
        bank: &signer,
        sp_code: String,
        sp_wallet_types: vector<u64>,
        sp_wallet_type_auths: vector<bool>,
        sp_accounts: vector<address>,
        sp_accounts_auth: vector<bool>,
        status: bool
    ) acquires Roles, ServiceProviders, WalletEventHandle {
        assert_is_bank(bank);

        let server_providers = borrow_global_mut<ServiceProviders>
(wallet_manage_address());
        assert!(simple_map::contains_key(&server_providers.service_provider_codes,
&sp_code), ESP_NOT_EXIST);

        update_service_provider_wallet_type(bank, sp_code, sp_wallet_types,
sp_wallet_type_auths);
        update_service_provider_status(bank, sp_code, status);
        set_service_provider_account(bank, sp_code, sp_accounts, sp_accounts_auth);
    }

    public entry fun register_wallet_by_bank(bank: &signer, sp_code: String,
wallet_address: address, wallet_type_id: u64, _data: String) acquires Roles,
WalletTypes, ServiceProviders, State, WalletEventHandle {
        assert_is_bank(bank);
        assert_register_status();
        assert_wallet_type(wallet_type_id);

        let sp_id: u64 = 0;

        if(string::length(&sp_code) >0){
            sp_id = get_service_provider_id(sp_code);
        };
        register_wallet_internal(signer::address_of(bank),
```

```
            sp_code,
            sp_id,
            wallet_address,
            wallet_type_id
        );
    }

    public entry fun register_wallet_by_sp(sp_account: &signer, wallet_address:
address, wallet_type_id: u64, _data: String) acquires Roles, WalletTypes,
ServiceProviders, State, WalletEventHandle {
        assert_register_status();
        assert_wallet_type(wallet_type_id);

        let (sp_id, sp_code) = assert_is_sp(sp_account);

        register_wallet_internal(signer::address_of(sp_account),
            sp_code,
            sp_id,
            wallet_address,
            wallet_type_id
        );
    }

    fun exists_wallet(wallet_address:address):bool acquires Roles {
        let wallet_resource_account_address =
get_wallet_resource_account_address(wallet_address);
        exists<WalletInfo>(wallet_resource_account_address)
    }

    fun register_wallet_internal(sender:address,sp_code: String, sp_id:u64,
wallet_address: address, wallet_type_id: u64) acquires Roles,WalletEventHandle {
        assert!(!exists_wallet(wallet_address),EWALLET_EXIST);
        let wallet_account_signer =
gen_wallet_resource_account_signer(wallet_address);
        move_to(&wallet_account_signer, WalletInfo {
            wallet_type_id,
            status: STATUS_ENABLED,
            is_use_default: true,
            deposit_limit: 0,
            daily_trans_limit: 0,
            per_trans_limit: 0,
            freeze_balance: 0,
            creator: sender,
            sp_id
        });

        move_to(&wallet_account_signer, WalletTrans {
            day_id:0,
            amount:0,
        });

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
```

```
        event::emit_event(
            &mut events.register_wallet_events,
            RegisterWallet { sender: sender, wallet_address, wallet_type_id, sp_code
});
    }

    public entry fun update_wallet_type(bank: &signer, wallet_address: address,
wallet_type_id: u64) acquires Roles, WalletInfo,WalletEventHandle {
        assert_is_bank(bank);
        check_wallet(wallet_address);

        let wallet_resource_account_address =
get_wallet_resource_account_address(wallet_address);
        let wallet_info = borrow_global_mut<WalletInfo>
(wallet_resource_account_address);
        wallet_info.wallet_type_id = wallet_type_id;

        let events = borrow_global_mut<WalletEventHandle>(wallet_manage_address());
        event::emit_event(
            &mut events.update_wallet_type_events,
            UpdateWalletType { sender: signer::address_of(bank), wallet_address,
wallet_type_id });
    }

    public(friend) fun check_allowed_transfer(from: address, to: address, amount:
u64): bool acquires Roles, WalletInfo,WalletTrans,WalletTypes {
        assert!(from != to, EFROM_TO_EQUAL);
        let repository_addr = repository_address();
        if (from != repository_addr && to != repository_addr) {
            assert_wallet_status(from);
            assert_wallet_status(to);
            let from_limit = get_wallet_limit_internal(from);
            assert!(from_limit.per_trans_limit >= amount, ETRANS_LIMIT);
            return is_over_limit(from, amount)

        } else if (from == repository_addr) {
            assert_wallet_status(to);
            return true
        } else if (to == repository_addr) {
            assert_wallet_status(from);
            return true
        };

        false
    }

    fun get_wallet_limit_internal(wallet_address: address): WalletLimit acquires
Roles,WalletInfo, WalletTypes {

        let wallet_resource_account_address =
get_wallet_resource_account_address(wallet_address);
```

```
        let wallet_info = borrow_global<WalletInfo>
(wallet_resource_account_address);

        if (wallet_info.is_use_default) {

            let wts = borrow_global<WalletTypes>(wallet_manage_address());
            let wt = simple_map::borrow(&wts.wallet_type_list,
&wallet_info.wallet_type_id);

            WalletLimit {
                deposit_limit: wt.deposit_limit,
                daily_trans_limit: wt.daily_trans_limit,
                per_trans_limit: wt.per_trans_limit,
                freeze_balance: wallet_info.freeze_balance
            }
        }else{
            WalletLimit {
                deposit_limit: wallet_info.deposit_limit,
                daily_trans_limit: wallet_info.daily_trans_limit,
                per_trans_limit: wallet_info.per_trans_limit,
                freeze_balance: wallet_info.freeze_balance
            }
        }

    }

    fun gen_wallet_resource_account_signer(wallet_address:address ):signer acquires
Roles {
        let wallet = borrow_global<Roles>(wallet_manage_address());
        let resource_signer =
account::create_signer_with_capability(&wallet.resource_signer_cap);

        let bytes = bcs::to_bytes(&wallet_address);
        vector::append(&mut bytes, WALLET_RESOURCE_SYMBOL);

        let (wallet_signer,_) =
account::create_resource_account(&resource_signer,bytes);
        wallet_signer
    }

    fun get_wallet_resource_account_address(wallet_address:address ):address
acquires Roles {
        let wallet = borrow_global<Roles>(wallet_manage_address());

        let bytes = bcs::to_bytes(&wallet_address);
        vector::append(&mut bytes, WALLET_RESOURCE_SYMBOL);

        let wallet_resource_account = account::create_resource_address(
            &account::get_signer_capability_address(&wallet.resource_signer_cap),
            bytes);

        wallet_resource_account
```

```
    }

    fun check_wallet(wallet_address:address) acquires Roles {
        assert!(exists_wallet(wallet_address),EWALLET_NOT_EXIST);
    }

    fun check_status_type(status: u64) {
        assert!(status == STATUS_ENABLED || status ==
STATUS_DISABLED,EWALLET_TYPE_STATUS);
    }

    #[view]
    public fun get_wallet_surplus_limit(account: address): u64 acquires
Roles,WalletInfo,WalletTrans, WalletTypes {

        let from_limit = get_wallet_limit_internal(account);

        let wallet_resource_account_address =
get_wallet_resource_account_address(account);
        let wallet_trans = borrow_global<WalletTrans>
(wallet_resource_account_address);

        if (wallet_trans.day_id != get_day_key()) {
            from_limit.daily_trans_limit
        } else {
            if (from_limit.daily_trans_limit <= wallet_trans.amount) {
                0
            }
            else {
                (from_limit.daily_trans_limit - wallet_trans.amount)
            }
        }
    }

    fun is_over_limit(account: address, amount: u64): bool acquires
Roles,WalletInfo,WalletTrans, WalletTypes {

        let supplus_limit = get_wallet_surplus_limit(account);
        assert!(supplus_limit >= amount, ETRANS_DAILY_LIMIT);

        set_wallet_daily_amount(account, amount);
        true
    }

    fun set_wallet_daily_amount(account: address, amount: u64) acquires
Roles,WalletTrans {
        let this_day_id = get_day_key();

        let wallet_resource_account_address =
get_wallet_resource_account_address(account);
```

```
        let wallet_trans = borrow_global_mut<WalletTrans>
(wallet_resource_account_address);

        if (wallet_trans.day_id != this_day_id) {
            wallet_trans.day_id = this_day_id;
            wallet_trans.amount = amount;
        } else {
            wallet_trans.amount = wallet_trans.amount + amount;
        }
    }

    fun get_service_provider_id(sp_code: String): u64 acquires ServiceProviders {
        let sp = borrow_global<ServiceProviders>(wallet_manage_address());

        assert!(simple_map::contains_key(&sp.service_provider_codes, &sp_code),
ESP_NOT_EXIST);

        let sp_id = simple_map::borrow(&sp.service_provider_codes, &sp_code);

        let sp_info = simple_map::borrow(&sp.service_providers, sp_id);

        assert!(sp_info.status, ESP_DISABLE);

        *sp_id
    }

    fun assert_exit_sp_wallet_type(sp_id: u64, wallet_type: u64) acquires
ServiceProviders {
        let sps = borrow_global<ServiceProviders>(wallet_manage_address());
        assert!(simple_map::contains_key(&sps.service_providers, &sp_id),
EUNAUTHORIZED);

        let sp = simple_map::borrow(&sps.service_providers, &sp_id);
        assert!(smart_vector::contains(&sp.wallet_types, &wallet_type),
ESP_WALLET_TYPE_UNAUTHORIZED);
    }

    fun assert_is_owner(owner: &signer) acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let owner_addr = signer::address_of(owner);
        assert!(roles.owner == owner_addr, EUNAUTHORIZED);
    }

    fun assert_is_owner_or_operation(owner: &signer) acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let owner_addr = signer::address_of(owner);
        assert!(roles.owner == owner_addr ||
smart_vector::contains(&roles.operation_accounts, &owner_addr), EUNAUTHORIZED);
    }

    fun assert_is_operation(account: &signer) acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
```

```
        let acc_addr = signer::address_of(account);
        assert!(smart_vector::contains(&roles.operation_accounts, &acc_addr),
EUNAUTHORIZED);
    }

    fun assert_is_bank(bank: &signer) acquires Roles {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let bank_addr = signer::address_of(bank);
        assert!(smart_vector::contains(&roles.bank_accounts, &bank_addr),
EUNAUTHORIZED);
    }

    fun assert_is_sp(sp_account: &signer): (u64, String) acquires Roles,
ServiceProviders {
        let roles = borrow_global<Roles>(wallet_manage_address());
        let sp_addr = signer::address_of(sp_account);
        assert!(simple_map::contains_key(&roles.sp_accounts, &sp_addr),
EUNAUTHORIZED);
        let sp_id = simple_map::borrow(&roles.sp_accounts, &sp_addr);

        let sps = borrow_global<ServiceProviders>(wallet_manage_address());
        assert!(simple_map::contains_key(&sps.service_providers, sp_id),
ESP_NOT_EXIST);

        let sp = simple_map::borrow(&sps.service_providers, sp_id);
        assert!(sp.status, ESP_DISABLE);

        (*sp_id, sp.code)
    }

    fun assert_not_paused() acquires State {
        let state = borrow_global<State>(wallet_manage_address());
        assert!(state.contract_status, EPAUSED);
    }

    fun assert_wallet_type(wallet_type: u64) acquires WalletTypes {
        let wts = borrow_global<WalletTypes>(wallet_manage_address());

        assert!(simple_map::contains_key(&wts.wallet_type_list, &wallet_type),
EWALLET_TYPE_INVALID);
        // assert!(smart_vector::contains(&wts.wallet_type_list,&wallet_type),
EWALLET_TYPE_INVALID);
    }

    fun assert_wallet_type_status(wallet_type: u64) acquires WalletTypes {
        let wts = borrow_global<WalletTypes>(wallet_manage_address());
        assert!(simple_map::contains_key(&wts.wallet_type_list, &wallet_type),
EWALLET_TYPE_INVALID);
        let wt = simple_map::borrow(&wts.wallet_type_list,&wallet_type);
        assert!(wt.status == STATUS_ENABLED, EWALLET_DISABLE);
    }
```

```
    fun assert_wallet_status(account: address) acquires Roles,WalletInfo,WalletTypes
{
        check_wallet(account);
        let wallet_resource_account_address =
get_wallet_resource_account_address(account);
        let wallet_info = borrow_global<WalletInfo>
(wallet_resource_account_address);
        assert!(wallet_info.status == STATUS_ENABLED, EWALLET_DISABLE);

        assert_wallet_type_status(wallet_info.wallet_type_id);
    }

    fun assert_register_status() acquires State {
        let state = borrow_global<State>(wallet_manage_address());
        assert!(state.register_status, ESP_REGISTER_DISABLE);
    }

    fun get_day_key(): u64 {
        let current_timestamp = timestamp::now_seconds();
        let days_since_epoch = current_timestamp / 86400;
        days_since_epoch
    }

    #[test_only]
    public fun init_for_test(wallet_signer: &signer) {
        init_module(wallet_signer);
    }
}
```

Register a Wallet

Method

```
public entry fun register_wallet_by_sp(sp_account: &signer, wallet_address: address,
wallet_type_id: u64, _data: String)
```

Parameter Description

```
wallet_address: the wallet address to be registered
wallet_type_id: the type of wallet
data: remarks
```

Event

```
#[event]
struct RegisterWallet has drop, store {
    sender: address,
    wallet_address: address,
    wallet_type_id: u64,
    sp_code: String
}
```

Get the List of Service Provider Wallet Type

Method

```
#[view]
public fun get_service_provider_wallet_types(sp_code: String): vector<u64>
```

Description

```
sp_code: Service Provider code
Returns a list of authorized wallet types
```

Get Wallet Type Information

Method

```
#[view]
public fun get_wallet_type_info(wallet_type_id: u64): (String, u64, u64, u64, u64)
```

Description

```
wallet_type_id: the wallet type ID

Return wallet type information:
name: the name of the wallet type
status: 0: Initialized 1: Enabled 2: Disabled
deposit_limit: the maximum balance limit
daily_trans_limit: the maximum daily transfer-out limit
per_trans_limit: the limit for each transfer-out
```

Get the Service Provider Code by Account

Method

```
#[view]
public fun get_account_sp_code(sp_account: address): String
```

Description

```
sp_account: the Service Provider account address
Returns the Service Provider code
```

Get Wallet Limit Information

Method

```
#[view]
public fun get_wallet_limit(wallet_address: address): (u64, u64, u64, u64)
```

Description

```
wallet_address: the wallet address

Return wallet limit information
per_trans_limit: the limit for each transfer-out
deposit_limit: the maximum balance limit
daily_trans_limit: the daily transfer-out limit
freeze_balance: the amount that is frozen
```

Get Transferable Token Balance

Method

```
#[view]
public fun get_wallet_surplus_limit(account: address): u64
```

Description

```
account: the wallet address
Return the transferable Token balance of the wallet on the day
```

Get Wallet Status

Method

```
#[view]
public fun get_wallet_status(account: address): bool
```

Description

```
account: the wallet address
Return the wallet status
1: Enabled
2: Disabled
```

Get the Wallet Type ID

Method

```
#[view]
public fun get_wallet_type(wallet_address: address): (u64)
```

Description

```
wallet_address: the wallet address
Returns the wallet type ID
```

Get the Service Provider ID by the Service Provider Code

Method

```
#[view]
fun get_service_provider_id(sp_code: String): u64
```

Description

```
sp_code: Service Provider code
Return the Service Provider ID
```

Query the SP Code

Method

```
#[view]
public fun get_account_sp_code(sp_account: address): String
```

Parameter Description

```
sp_account: the SP account address
Return the SP code
```

2.2.3 Contract Interface Status Code

Exception Code	Parameter Description	Remarks
10001	No permission	The caller does not have permission to call the method.
10002	The address cannot be empty	Wallet address/contract address cannot be empty.
10003	nvalid operation/No operation required	No change detected. The new value is the same as the original value.
10004	The amount of Tokens to be operated must be greater than 0	
11001	The account address for the operation already exists	Used to verify the method of adding the address of a bank account or operator account.
11002	The operator account address does not exist	Used to verify the deletion of a bank or operator account address.
11003	Incorrect wallet type	
11004	Incorrect wallet status type	Verify the input parameter of the wallet status type
11005	Wallet status has been disabled	
11006	The wallet has been cancelled or is not activated yet	
11007	From address and to address cannot be the same	
11008	The transfer amount exceeds the limit	
11009	The transfer amount is insufficient or has exceeded today's limit	
11010	Unavailable wallet type	

Exception Code	Parameter Description	Remarks
11011	The SP organization name has been registered	Change the SP name
11012	The SP organization is not registered	Register the SP organization first, or use a registered SP name.
11013	The number of parameters in the array does not match or is 0	Ensure parameter counts in lists match.
11014	This account has already been registered and is used by another SP	
11015	The sender does not have permission to register the wallet	
11016	When calling a contract to register the wallet, the contract address and the signer's address do not belong to the same SP organization	
11017	The wallet type is not authorized to register a wallet for this SP organization	
11018	The SP organization status has been disabled	
11019	The authorization amount to be decreased cannot be greater than the original value.	
11020	The SP registration has been disabled	Contact your bank to activate SP registration.
11022	Account balance has reached the limit	Reduce the transfer amount to proceed.
11023	Wallet type already exists	
12001	Insufficient account balance	Balance in the transfer account is too low to complete transaction.
12002	Insufficient authorization amount	Authorized amount must be greater than the transfer amount.
12003	Transfer verification failed	Failed to verify the transfer limit.
12004	Account non-frozen balance is insufficient	Available balance in the account is insufficient for transfer.

Exception Code	Parameter Description	Remarks
12005	Account limit excess	The amount of transfer-in Tokens exceeds the account limit.
13001	The exchange rate needs to be greater than 0	
13002	Transfer failed	
11404	Contract disabled	